```
--------------------------------------------------------------------------------
-- Module : rgb_kmeans_cluster_engine
-- Author      : Sakinder
-- Description :
-- Refined version of original clustering module.
--
-- Function:
--   Performs RGB color clustering for real-time image processing.
--   Incoming pixel RGB values are compared with centroid lookup tables
--   and assigned to the nearest cluster.
--
-- Key Processing Steps:
--   1. Receive RGB pixel stream
--   2. Compare pixel against K cluster centroids
--   3. Compute distance / similarity metric
--   4. Select closest centroid
--   5. Output clustered RGB value
--
-- Notes:
--   Signal names simplified for readability.
--   Functional behavior unchanged.
--------------------------------------------------------------------------------
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;
use work.fixed_pkg.all;
use work.float_pkg.all;
use work.constants_package.all;
use work.vpf_records.all;
use work.ports_package.all;
use work.vfp_pkg.all;
entity rgb_kmeans_cluster_engine is
generic (
    i_data_width   : integer := 8);
port (
    clk            : in std_logic;
    rst_n          : in std_logic;
    pixel_in_rgb           : in channel;
    centroid_lut_select : in natural;
    centroid_lut_in       : in std_logic_vector(23 downto 0);
    centroid_lut_out      : out std_logic_vector(31 downto 0);
    k_ind_w        : in natural;
    k_ind_r        : in natural;
    pixel_out_rgb           : out channel);
end rgb_kmeans_cluster_engine;
architecture arch of rgb_kmeans_cluster_engine is
    signal rgbSyncEol    : std_logic_vector(31 downto 0) := x"00000000";
    signal rgbSyncSof    : std_logic_vector(31 downto 0) := x"00000000";
    signal rgbSyncEof    : std_logic_vector(31 downto 0) := x"00000000";
    signal rgbSyncValid  : std_logic_vector(31 downto 0) := x"00000000";
    signal k_rgb         : rgb_k_lut(0 to 30);
    constant k_rgb_lut_1_l             : rgb_k_range(0 to 30) := (
    (  0,  0,  0),
    (255,240,230),
    (240,220,210),
    (240,220,170),
    (240,210,150),
    (240,210,180),
    (240,200,140),
    (240,200,160),
    (240,180,150),
    (240,150,140),
    (180,140,160),
    (180,160,140),
    (180,170,120),
    (180,145,100),
    (180,140, 90),
    (180,130, 80),
    (150,140,130),
    (150,120,110),
    (150,100, 80),
    (150, 80, 50),
    (130,120,100),
    (130,100, 80),
```

```
75        (130, 80, 60),
76        (130, 60, 30),
77        (120,110,100),
78        (120,100, 90),
79        (120, 80, 70),
80        (120, 60, 30),
81        (100, 90, 70),
82        (100, 70, 40),
83        (100, 50, 10));
84        signal k_rgb_lut_2_l              : rgb_k_range(0 to 30) := (
85        (  0,  0,  6),
86        ( 90, 90,  0),
87        ( 90, 50, 20),
88        ( 90, 40, 10),
89        ( 90, 40, 10),
90        ( 90, 30,  0),
91        ( 90, 30,  0),
92        ( 90, 20,  0),
93        ( 80, 80,  0),
94        ( 80, 50, 20),
95        ( 80, 40, 10),
96        ( 80, 40, 10),
97        ( 80, 30,  0),
98        ( 80, 30,  0),
99        ( 70, 70,  0),
100       ( 70, 50, 15),
101       ( 70, 30, 15),
102       ( 70, 10,  0),
103       ( 60, 60,  0),
104       ( 60, 30, 20),
105       ( 60, 20, 10),
106       ( 60, 10,  0),
107       ( 40, 40,  0),
108       ( 40, 20, 10),
109       ( 40, 10,  5),
110       ( 40, 10,  0),
111       ( 30, 30,  0),
112       ( 30, 20, 15),
113       ( 30, 15,  0),
114       ( 30, 10,  0),
115       ( 20, 10,  0));
116       signal k_rgb_lut_2ll              : rgb_k_range(0 to 30) := (
117       (  0,  0,  6),
118       ( 90, 90,  0),
119       ( 90, 50, 20),
120       ( 90, 40, 10),
121       ( 90, 40, 10),
122       ( 90, 30,  0),
123       ( 90, 30,  0),
124       ( 90, 20,  0),
125       ( 80, 80,  0),
126       ( 80, 50, 20),
127       ( 80, 40, 10),
128       ( 80, 40, 10),
129       ( 80, 30,  0),
130       ( 80, 30,  0),
131       ( 70, 70,  0),
132       ( 70, 50, 15),
133       ( 70, 30, 15),
134       ( 70, 10,  0),
135       ( 60, 60,  0),
136       ( 60, 30, 20),
137       ( 60, 20, 10),
138       ( 60, 10,  0),
139       ( 40, 40,  0),
140       ( 40, 20, 10),
141       ( 40, 10,  5),
142       ( 40, 10,  0),
143       ( 30, 30,  0),
144       ( 30, 20, 15),
145       ( 30, 15,  0),
146       ( 30, 10,  0),
147       ( 20, 10,  0));
148       constant k_rgb_lut_22l              : rgb_k_range(0 to 30) := (
```

```
149        (  0,  0,  6),
150        ( 90, 90,  0),
151        ( 90, 50, 20),
152        ( 90, 40, 10),
153        ( 90, 40, 10),
154        ( 90, 30,  0),
155        ( 90, 30,  0),
156        ( 90, 20,  0),
157        ( 80, 80,  0),
158        ( 80, 50, 20),
159        ( 80, 40, 10),
160        ( 80, 40, 10),
161        ( 80, 30,  0),
162        ( 80, 30,  0),
163        ( 70, 70,  0),
164        ( 70, 50, 15),
165        ( 70, 30, 15),
166        ( 70, 10,  0),
167        ( 60, 60,  0),
168        ( 60, 30, 20),
169        ( 60, 20, 10),
170        ( 60, 10,  0),
171        ( 40, 40,  0),
172        ( 40, 20, 10),
173        ( 40, 10,  5),
174        ( 40, 10,  0),
175        ( 30, 30,  0),
176        ( 30, 20, 15),
177        ( 30, 15,  0),
178        ( 30, 10,  0),
179        ( 20, 10,  0));
180        constant k_rgb_lut_0_l              : rgb_k_range(0 to 30) := (
181        ( 0,  0,  6),
182        (255,255,255),
183        (250,250,250),
184        (240,240,240),
185        (230,230,230),
186        (220,220,220),
187        (210,210,210),
188        (200,200,200),
189        (190,190,190),
190        (180,180,180),
191        (170,170,170),
192        (160,160,160),
193        (150,150,150),
194        (140,140,140),
195        (130,130,130),
196        (120,120,120),
197        (110,110,110),
198        (100,100,100),
199        ( 90, 90, 90),
200        ( 80, 80, 80),
201        ( 70, 70, 70),
202        ( 60, 60, 60),
203        ( 50, 50, 50),
204        ( 40, 40, 40),
205        ( 30, 30, 30),
206        ( 20, 20, 20),
207        ( 10, 10, 10),
208        (  0,  0,  0),
209        (255,255,255),
210        (150,150,150),
211        (100,100,100));
212        signal k_rgb_lut_4_l                : rgb_k_range(0 to 30) := (
213        (  0,  0,  0),
214        (240,200,  0),
215        (240,180,  0),
216        (230,180,  0),
217        (230,170,  0),
218        (230,160,  0),
219        (230,180,  0),
220        (200,160,  0),
221        (200,150,  0),
222        (200,140,  0),
```

```
223        (200,130,  0),
224        (200,100,  0),
225        (160,160,  0),
226        (160,120, 70),
227        (160,110, 50),
228        (160,100, 40),
229        (160, 90, 30),
230        (160, 50, 10),
231        (150, 80, 20),
232        (140,140,  0),
233        (140,100, 40),
234        (140, 80, 40),
235        (140, 60,  0),
236        (120,120,  0),
237        (120, 50,  0),
238        (120, 80,  0),
239        (120, 70,  0),
240        (100,100,  0),
241        (100, 50,  0),
242        (100, 90,  0),
243        (100, 25,  0));
244        signal k_rgb_lut_4ll                : rgb_k_range(0 to 30) := (
245        (  0,  0,  0),
246        (240,200,  0),
247        (240,180,  0),
248        (230,180,  0),
249        (230,170,  0),
250        (230,160,  0),
251        (230,180,  0),
252        (200,160,  0),
253        (200,150,  0),
254        (200,140,  0),
255        (200,130,  0),
256        (200,100,  0),
257        (160,160,  0),
258        (160,120, 70),
259        (160,110, 50),
260        (160,100, 40),
261        (160, 90, 30),
262        (160, 50, 10),
263        (150, 80, 20),
264        (140,140,  0),
265        (140,100, 40),
266        (140, 80, 40),
267        (140, 60,  0),
268        (120,120,  0),
269        (120, 50,  0),
270        (120, 80,  0),
271        (120, 70,  0),
272        (100,100,  0),
273        (100, 50,  0),
274        (100, 90,  0),
275        (100, 25,  0));
276        signal k_rgb_lut_3ll                : rgb_k_range(0 to 30) := (
277        (  0,  0,  0),
278        (240,200,  0),
279        (240,180,  0),
280        (230,180,  0),
281        (230,170,  0),
282        (230,160,  0),
283        (230,180,  0),
284        (200,160,  0),
285        (200,150,  0),
286        (200,140,  0),
287        (200,130,  0),
288        (200,100,  0),
289        (160,160,  0),
290        (160,120, 70),
291        (160,110, 50),
292        (160,100, 40),
293        (160, 90, 30),
294        (160, 50, 10),
295        (150, 80, 20),
296        (140,140,  0),
```

```
297        (140,100, 40),
298        (140, 80, 40),
299        (140, 60,  0),
300        (120,120,  0),
301        (120, 50,  0),
302        (120, 80,  0),
303        (120, 70,  0),
304        (100,100,  0),
305        (100, 50,  0),
306        (100, 90,  0),
307        (100, 25,  0));
308        signal k_rgb_lut_6ll                : rgb_k_range(0 to 30) := (
309        (  0,  0,  0),
310        (240,200,  0),
311        (240,180,  0),
312        (230,180,  0),
313        (230,170,  0),
314        (230,160,  0),
315        (230,180,  0),
316        (200,160,  0),
317        (200,150,  0),
318        (200,140,  0),
319        (200,130,  0),
320        (200,100,  0),
321        (160,160,  0),
322        (160,120, 70),
323        (160,110, 50),
324        (160,100, 40),
325        (160, 90, 30),
326        (160, 50, 10),
327        (150, 80, 20),
328        (140,140,  0),
329        (140,100, 40),
330        (140, 80, 40),
331        (140, 60,  0),
332        (120,120,  0),
333        (120, 50,  0),
334        (120, 80,  0),
335        (120, 70,  0),
336        (100,100,  0),
337        (100, 50,  0),
338        (100, 90,  0),
339        (100, 25,  0));
340        signal k_rgb_lut_3_l                : rgb_k_range(0 to 30) := (
341        (  0,  0,  0),
342        (240,200,  0),
343        (240,180,  0),
344        (230,180,  0),
345        (230,170,  0),
346        (230,160,  0),
347        (230,180,  0),
348        (200,160,  0),
349        (200,150,  0),
350        (200,140,  0),
351        (200,130,  0),
352        (200,100,  0),
353        (160,160,  0),
354        (160,120, 70),
355        (160,110, 50),
356        (160,100, 40),
357        (160, 90, 30),
358        (160, 50, 10),
359        (150, 80, 20),
360        (140,140,  0),
361        (140,100, 40),
362        (140, 80, 40),
363        (140, 60,  0),
364        (120,120,  0),
365        (120, 50,  0),
366        (120, 80,  0),
367        (120, 70,  0),
368        (100,100,  0),
369        (100, 50,  0),
370        (100, 90,  0),
```

```
371         (100, 25,  0));
372         signal k_rgb_lut_6_l                : rgb_k_range(0 to 30) := (
373         (  0,  0,  0),
374         (240,200,  0),
375         (240,180,  0),
376         (230,180,  0),
377         (230,170,  0),
378         (230,160,  0),
379         (230,180,  0),
380         (200,160,  0),
381         (200,150,  0),
382         (200,140,  0),
383         (200,130,  0),
384         (200,100,  0),
385         (160,160,  0),
386         (160,120, 70),
387         (160,110, 50),
388         (160,100, 40),
389         (160, 90, 30),
390         (160, 50, 10),
391         (150, 80, 20),
392         (140,140,  0),
393         (140,100, 40),
394         (140, 80, 40),
395         (140, 60,  0),
396         (120,120,  0),
397         (120, 50,  0),
398         (120, 80,  0),
399         (120, 70,  0),
400         (100,100,  0),
401         (100, 50,  0),
402         (100, 90,  0),
403         (100, 25,  0));
404
405         constant k_rgb_lut_41l              : rgb_k_range(0 to 30) := (
406         (  0,  0,  0),
407         (255,240,150),
408         (240,230,140),
409         (220,200,120),
410         (255,230, 75),
411         (240,220, 75),
412         (220,120, 75),
413         (200,180,158),
414         (180,160,128),
415         (160,140,118),
416         (140,120, 98),
417         (120,100, 88),
418         (100, 80, 78),
419         (255,210,128),
420         (240,190,120),
421         (220,170,110),
422         (200,150,100),
423         (180,130, 80),
424         (160,110, 60),
425         (140, 90, 40),
426         (120, 70, 20),
427         (100, 50,  0),
428         (255,150, 75),
429         (240,100, 75),
430         (220, 80, 75),
431         (200,140,100),
432         (180,120, 80),
433         (160,100, 60),
434         (140, 80, 40),
435         (120, 60, 20),
436         (100, 40,  0));
437         constant k_rgb_lut_42l              : rgb_k_range(0 to 30) := (
438         (  0,  0,  0),
439         (255,251,242),
440         (255,241,232),
441         (255,231,202),
442         (255,221,182),
443         (255,211,122),
444         (255,201,142),
```

```
445        (255,191,132),
446        (255,181,122),
447        (255,171,112),
448        (230,191,172),
449        (230,181,162),
450        (230, 81, 62),
451        (220,181,142),
452        (220,171,132),
453        (200, 51, 12),
454        (200,181, 92),
455        (180,161,132),
456        (180,151,122),
457        (170,151,112),
458        (170,141,102),
459        (150, 31, 22),
460        (150, 71, 42),
461        (130, 91, 62),
462        (130, 81, 52),
463        (120, 81, 42),
464        (120, 71, 32),
465        (100, 51, 22),
466        (100, 51, 22),
467        (100, 51, 22),
468        ( 40, 21, 12));
469        constant k_rgb_lut_43l             : rgb_k_range(0 to 30) := (
470        (  0,  0,  0),
471        (240,230,140),
472        (240,230,140),
473        (240,230,140),
474        (240,220, 75),
475        (240,220, 75),
476        (240,220, 75),
477        (200,180,158),
478        (180,160,128),
479        (180,160,128),
480        (180,160,128),
481        (120,100, 88),
482        (120,100, 88),
483        (240,190,120),
484        (240,190,120),
485        (240,190,120),
486        (200,150,100),
487        (200,150,100),
488        (160,110, 60),
489        (160,110, 60),
490        (120, 70, 20),
491        (120, 70, 20),
492        (255,240,230),
493        (246,216,192),
494        (236,192,145),
495        (207,150, 95),
496        (161,110, 75),
497        (127, 68, 34),
498        (140, 80, 40),
499        (120, 60, 20),
500        (100, 40,  0));
501        constant k_rgb_lut_44l             : rgb_k_range(0 to 30) := (
502        (  0,  0,  0),
503        (240,230,140),
504        (240,230,140),
505        (240,230,140),
506        (240,230,140),
507        (240,230,140),
508        (240,230,140),
509        (180,160,128),
510        (180,160,128),
511        (180,160,128),
512        (180,160,128),
513        (180,160,128),
514        (120,100, 88),
515        (120,100, 88),
516        (120,100, 88),
517        (120,100, 88),
518        (200,150,100),
```

```
519        (200,150,100),
520        (200,150,100),
521        (200,150,100),
522        (120, 70, 20),
523        (120, 70, 20),
524        (255,240,230),
525        (246,216,192),
526        (236,192,145),
527        (207,150, 95),
528        (161,110, 75),
529        (127, 68, 34),
530        (140, 80, 40),
531        (120, 60, 20),
532        (100, 40,  0));
533        constant k_rgb_lut_45l             : rgb_k_range(0 to 30) := (
534        (  0,  0,  0),
535        (255,240,150),
536        (220,200,120),
537        (220,200,120),
538        (220,200,120),
539        (220,200,120),
540        (220,120, 75),
541        (160,140,118),
542        (160,140,118),
543        (160,140,118),
544        (100, 80, 78),
545        (100, 80, 78),
546        (100, 80, 78),
547        (255,240,230),
548        (255,240,230),
549        (246,216,192),
550        (236,192,145),
551        (207,150, 95),
552        (161,110, 75),
553        (127, 68, 34),
554        (120, 70, 20),
555        (100, 50,  0),
556        (255,150, 75),
557        (240,100, 75),
558        (220, 80, 75),
559        (200,140,100),
560        (180,120, 80),
561        (160,100, 60),
562        (140, 80, 40),
563        (120, 60, 20),
564        (100, 40,  0));
565        constant k_rgb_lut_46l             : rgb_k_range(0 to 30) := (
566        (  0,  0,  0),
567        (255,240,  0),
568        (240,230,  0),
569        (220,200,  0),
570        (255,230,  0),
571        (255,240,  0),
572        (246,216,  0),
573        (236,192,  0),
574        (220,120,  0),
575        (207,150,  0),
576        (165,100,  0),
577        (127, 68,  0),
578        (209,168,  0),
579        (200,180,  0),
580        (180,160,  0),
581        (160,140,  0),
582        (140,120,  0),
583        (120,100,  0),
584        (100, 80,  0),
585        (240,190,  0),
586        (200,150,  0),
587        (180,130,  0),
588        (160,110,  0),
589        (140, 90,  0),
590        (240,100,  0),
591        (220, 80,  0),
592        (180,120,  0),
```

```
593         (160,100,  0),
594         (140, 80,  0),
595         (120, 60,  0),
596         (100, 40,  0));
597         constant k_rgb_lut_47l              : rgb_k_range(0 to 30) := (
598         (  0,  0,  0),
599         (255,251,242),
600         (255,241,232),
601         (255,231,202),
602         (255,221,182),
603         (209,168,138),
604         (255,201,142),
605         (255,191,132),
606         (255,181,122),
607         (255,240,230),
608         (255,240,230),
609         (246,216,192),
610         (236,192,145),
611         (220,120, 75),
612         (207,150, 95),
613         (200,150,100),
614         (127, 68, 34),
615         (209,168,138),
616         (180,151,122),
617         (170,151,112),
618         (170,141,102),
619         (200,150,100),
620         (150, 71, 42),
621         (130, 91, 62),
622         (130, 81, 52),
623         (120, 81, 42),
624         (120, 71, 32),
625         (100, 51, 22),
626         (100, 51, 22),
627         (100, 51, 22),
628         ( 40, 21, 12));
629         constant k_rgb_lut_48l              : rgb_k_range(0 to 30) := (
630         (  0,  0,  0),
631         (240,230,140),
632         (240,220, 75),
633         (240,220, 75),
634         (255,240,230),
635         (255,240,230),
636         (246,216,192),
637         (236,192,145),
638         (220,120, 75),
639         (207,150, 95),
640         (165,100, 80),
641         (127, 68, 34),
642         (209,168,138),
643         (240,190,120),
644         (240,190,120),
645         (240,190,120),
646         (200,150,100),
647         (200,150,100),
648         (160,110, 60),
649         (160,110, 60),
650         (120, 70, 20),
651         (120, 70, 20),
652         (255,240,230),
653         (246,216,192),
654         (236,192,145),
655         (207,150, 95),
656         (161,110, 75),
657         (127, 68, 34),
658         (140, 80, 40),
659         (120, 60, 20),
660         (100, 40,  0));
661         constant k_rgb_lut_49l              : rgb_k_range(0 to 30) := (
662         (  0,  0,  0),
663         (240,230,140),
664         (240,230,140),
665         (240,230,140),
666         (240,230,140),
```

```
667         (240,230,140),
668         (240,230,140),
669         (180,160,128),
670         (180,160,128),
671         (180,160,128),
672         (180,160,128),
673         (180,160,128),
674         (120,100, 88),
675         (120,100, 88),
676         (120,100, 88),
677         (120,100, 88),
678         (200,150,100),
679         (200,150,100),
680         (200,150,100),
681         (200,150,100),
682         (120, 70, 20),
683         (120, 70, 20),
684         (255,240,230),
685         (246,216,192),
686         (236,192,145),
687         (207,150, 95),
688         (161,110, 75),
689         (127, 68, 34),
690         (140, 80, 40),
691         (120, 60, 20),
692         (100, 40,  0));
693         constant k_rgb_lut_50l              : rgb_k_range(0 to 30) := (
694         (  0,  0,  0),
695         (255,240,150),
696         (220,200,120),
697         (220,200,120),
698         (220,200,120),
699         (209,168,138),
700         (220,120, 75),
701         (165,100, 80),
702         (165,100, 80),
703         (165,100, 80),
704         (100, 80, 78),
705         (100, 80, 78),
706         (100, 80, 78),
707         (255,240,230),
708         (255,240,230),
709         (246,216,192),
710         (236,192,145),
711         (207,150, 95),
712         (161,110, 75),
713         (127, 68, 34),
714         (120, 70, 20),
715         (100, 50,  0),
716         (255,150, 75),
717         (240,100, 75),
718         (220, 80, 75),
719         (200,140,100),
720         (180,120, 80),
721         (160,100, 60),
722         (140, 80, 40),
723         (120, 60, 20),
724         (100, 40,  0));
725         constant k_rgb_lut_51l              : rgb_k_range(0 to 30) := (
726         (  0,  0,  0),
727         (150,140,130),
728         (148,137,120),
729         (146,134,110),
730         (144,131,100),
731         (142,128, 90),
732         (140,125, 80),
733         (138,122, 70),
734         (136,119, 60),
735         (134,116, 50),
736         (132,113, 40),
737         (130,110, 30),
738         (128,107, 20),
739         (126,104, 10),
740         (124,101, 0),
```

```
741        (122, 98, 10),
742        (120, 95, 20),
743        (118, 92, 30),
744        (116, 89, 40),
745        (114, 86, 50),
746        (112, 83, 60),
747        (110, 80, 70),
748        (108, 77, 10),
749        (106, 74, 20),
750        (104, 71, 30),
751        (102, 68, 40),
752        (100, 65, 50),
753        (100, 62, 20),
754        (100, 59, 30),
755        (100, 56, 40),
756        (100, 53, 0));
757        constant k_rgb_lut_52l              : rgb_k_range(0 to 30) := (
758        (  0,  0,  0),
759        (240,200,150),
760        (200,160,120),
761        (170,110, 60),
762        (144,131,100),
763        (142,128, 90),
764        (140,125, 80),
765        (138,122, 70),
766        (136,119, 60),
767        (134,116, 50),
768        (132,113, 40),
769        (130,110, 30),
770        (128,107, 20),
771        (126,104, 10),
772        (124,101, 0),
773        (122, 98, 10),
774        (120, 95, 20),
775        (118, 92, 30),
776        (116, 89, 40),
777        (114, 86, 50),
778        (112, 83, 60),
779        (110, 80, 70),
780        (108, 77, 10),
781        (106, 74, 20),
782        (104, 71, 30),
783        (102, 68, 40),
784        (100, 65, 50),
785        (100, 62, 20),
786        (100, 59, 30),
787        (100, 56, 40),
788        (100, 53, 0));
789        constant k_rgb_lut_53l              : rgb_k_range(0 to 30) := (
790        (  0,  0,  0),
791        (240,200,150),
792        (200,160,120),
793        (170,110, 60),
794        (144,131,100),
795        (144,131,100),
796        (144,131,100),
797        (144,131,100),
798        (128,107, 20),
799        (128,107, 20),
800        (128,107, 20),
801        (128,107, 20),
802        (128,107, 20),
803        (112, 83, 60),
804        (112, 83, 60),
805        (112, 83, 60),
806        (112, 83, 60),
807        (112, 83, 60),
808        (112, 83, 60),
809        (112, 83, 60),
810        (112, 83, 60),
811        (110, 80, 70),
812        (108, 77, 10),
813        (106, 74, 20),
814        (104, 71, 30),
```

```
815        (102, 68, 40),
816        (100, 65, 50),
817        (100, 62, 20),
818        (100, 56, 40),
819        (100, 56, 40),
820        (100, 56, 40));
821        constant k_rgb_lut_54l             : rgb_k_range(0 to 30) := (
822        (  0,  0,  0),
823        (240,200,150),
824        (200,160,120),
825        (170,110, 60),
826        (144,131,100),
827        (144,131,100),
828        (144,131,100),
829        (144,131,100),
830        (128,107, 20),
831        (128,107, 20),
832        (128,107, 20),
833        (128,107, 20),
834        (128,107, 20),
835        (112, 83, 60),
836        (112, 83, 60),
837        (112, 83, 60),
838        (112, 83, 60),
839        (112, 83, 60),
840        (112, 83, 60),
841        (112, 83, 60),
842        (112, 83, 60),
843        (100, 56, 40),
844        (100, 56, 40),
845        (100, 56, 40),
846        (100, 56, 40),
847        (100, 56, 40),
848        (100, 56, 40),
849        (100, 56, 40),
850        (100, 56, 40),
851        (100, 56, 40),
852        (100, 56, 40));
853        constant k_rgb_lut_55l             : rgb_k_range(0 to 30) := (
854        (  0,  0,  0),
855        (240,200,150),
856        (144,131,100),
857        (144,131,100),
858        (144,131,100),
859        (144,131,100),
860        (144,131,100),
861        (144,131,100),
862        (128,107, 20),
863        (128,107, 20),
864        (128,107, 20),
865        (128,107, 20),
866        (128,107, 20),
867        (112, 83, 60),
868        (112, 83, 60),
869        (112, 83, 60),
870        (112, 83, 60),
871        (112, 83, 60),
872        (112, 83, 60),
873        (112, 83, 60),
874        (112, 83, 60),
875        (100, 56, 40),
876        (100, 56, 40),
877        (100, 56, 40),
878        (100, 56, 40),
879        (100, 56, 40),
880        (100, 56, 40),
881        (100, 56, 40),
882        (100, 56, 40),
883        (100, 56, 40),
884        (100, 56, 40));
885        constant k_rgb_lut_56l             : rgb_k_range(0 to 30) := (
886        (  0,  0,  0),
887        (240,200,  0),
888        (240,180,  0),
```

```
889        (230,180,  0),
890        (230,170,  0),
891        (230,160,  0),
892        (230,180,  0),
893        (200,160,  0),
894        (200,150,  0),
895        (200,140,  0),
896        (200,130,  0),
897        (200,200,  0),
898        (160,160,  0),
899        (140,140,  0),
900        (120,120,  0),
901        (100,100,  0),
902        (160,120, 70),
903        (160,110, 50),
904        (160,100, 40),
905        (160, 90, 30),
906        (160, 80, 20),
907        (160, 70, 10),
908        (140,100, 40),
909        (140,100, 30),
910        (140,100, 20),
911        (140,100, 10),
912        (120, 80,  0),
913        (120, 70,  0),
914        (100, 90,  0),
915        (100, 80,  0),
916        (100, 70,  0));
917        constant k_rgb_lut_57l             : rgb_k_range(0 to 30) := (
918        (  0,  0,  0),
919        (240,240,  0),
920        (240,200,  0),
921        (240,180,  0),
922        (230,230,  0),
923        (230,160,  0),
924        (230,180,  0),
925        (200,160,  0),
926        (200,150,  0),
927        (200,140,  0),
928        (200,130,  0),
929        (200,200,  0),
930        (160,160,  0),
931        (140,140,  0),
932        (120,120,  0),
933        (100,100,  0),
934        (160,120,  0),
935        (160,110,  0),
936        (160,100,  0),
937        (160, 90,  0),
938        (160, 80,  0),
939        (160, 70,  0),
940        (140,100,  0),
941        (140,100,  0),
942        (140,100,  0),
943        (140,100,  0),
944        (120, 80,  0),
945        (120, 70,  0),
946        (100, 90,  0),
947        (100, 80,  0),
948        (100, 70,  0));
949        constant k_rgb_lut_58l             : rgb_k_range(0 to 30) := (
950        (  0,  0,  0),
951        (240,240,  0),
952        (240,200,  0),
953        (240,180,  0),
954        (230,230,  0),
955        (230,160,  0),
956        (230,180,  0),
957        (255,240,230),-- required brown colors
958        (246,216,192),-- required brown colors
959        (236,192,145),-- required brown colors
960        (220,120, 75),-- required brown colors
961        (207,150, 95),-- required brown colors
962        (165,100, 80),-- required brown colors
```

```
963        (127, 68, 34),-- required brown colors
964        (209,168,138),-- required brown colors
965        (200,150,100),-- required brown colors
966        (200,100,100),
967        (200,200,  0),
968        (160,160,  0),
969        (120,120,  0),
970        (100,100,  0),
971        (160,120,  0),
972        (160,110,  0),
973        (160,100,  0),
974        (160, 90,  0),
975        (160, 80,  0),
976        (140,100,  0),
977        (100, 50, 50),
978        (120, 80,  0),
979        (100, 90,  0),
980        (100, 80,  0));
981        constant k_rgb_lut_59l             : rgb_k_range(0 to 30) := (
982        (  0,  0,  0),
983        (240,200,  0),
984        (240,180,  0),
985        (230,180,  0),
986        (230,170,  0),
987        (230,160,  0),
988        (230,180,  0),
989        (200,160,  0),
990        (200,150,  0),
991        (200,140,  0),
992        (200,130,  0),
993        (200,100,  0),
994        (160,160,  0),
995        (160,120, 70),
996        (160,110, 50),
997        (160,100, 40),
998        (160, 90, 30),
999        (160, 50, 10),
1000       (150, 80, 20),
1001       (140,140,  0),
1002       (140,100, 40),
1003       (140, 80, 40),
1004       (140, 60,  0),
1005       (120,120,  0),
1006       (120, 50,  0),
1007       (120, 80,  0),
1008       (120, 70,  0),
1009       (100,100,  0),
1010       (100, 50,  0),
1011       (100, 90,  0),
1012       (100, 25,  0));
1013
1014
1015
1016       signal k1_rgb                : rgb_k_lut(0 to 30);
1017       signal k2_rgb                : rgb_k_lut(0 to 30);
1018       signal k3_rgb                : rgb_k_lut(0 to 30);
1019       signal k4_rgb                : rgb_k_lut(0 to 30);
1020       signal k5_rgb                : rgb_k_lut(0 to 30);
1021       signal k6_rgb                : rgb_k_lut(0 to 30);
1022       signal k7_rgb                : rgb_k_lut(0 to 30);
1023       signal k8_rgb                : rgb_k_lut(0 to 30);
1024       signal k9_rgb                : rgb_k_lut(0 to 30);
1025       signal k10rgb                : rgb_k_lut(0 to 30);
1026       signal k11rgb                : rgb_k_lut(0 to 30);
1027       signal k12rgb                : rgb_k_lut(0 to 30);
1028       signal k13rgb                : rgb_k_lut(0 to 30);
1029       signal k14rgb                : rgb_k_lut(0 to 30);
1030       signal k15rgb                : rgb_k_lut(0 to 30);
1031       signal k16rgb                : rgb_k_lut(0 to 30);
1032       signal k17rgb                : rgb_k_lut(0 to 30);
1033       signal k18rgb                : rgb_k_lut(0 to 30);
1034       signal k19rgb                : rgb_k_lut(0 to 30);
1035       signal k20rgb                : rgb_k_lut(0 to 30);
1036       signal k21rgb                : rgb_k_lut(0 to 30);
```

```
1037        signal k22rgb                 : rgb_k_lut(0 to 30);
1038        signal k23rgb                 : rgb_k_lut(0 to 30);
1039        signal k24rgb                 : rgb_k_lut(0 to 30);
1040        signal k25rgb                 : rgb_k_lut(0 to 30);
1041        signal k26rgb                 : rgb_k_lut(0 to 30);
1042        signal k27rgb                 : rgb_k_lut(0 to 30);
1043        signal rgb                    : k_val_rgb(0 to 30);
1044        signal thr1                   : thr_record;
1045        signal thr2                   : thr_record;
1046        signal thr3                   : thr_record;
1047        signal thr4                   : thr_record;
1048        signal thr5                   : thr_record;
1049        signal threshold              : integer;
1050        signal threshold_lms_1        : integer;
1051        signal threshold_lms_2        : integer;
1052        signal threshold_lms_3        : integer;
1053        signal threshold_lms_4        : integer;
1054        signal threshold_lms_5        : integer;
1055        signal threshold_lms_6        : integer;
1056        signal threshold_lms_7        : integer;
1057        signal threshold_lms_8        : integer;
1058        signal threshold_lms_9        : integer;
1059        signal threshold_lms10        : integer;
1060        signal threshold_lms11        : integer;
1061        signal threshold_lms12        : integer;
1062        signal threshold_lms13        : integer;
1063        signal threshold_lms14        : integer;
1064        signal threshold_lms15        : integer;
1065        signal threshold_lms16        : integer;
1066        signal threshold_lms17        : integer;
1067        signal threshold_lms18        : integer;
1068        signal threshold_lms19        : integer;
1069        signal threshold_lms20        : integer;
1070        signal threshold_lms21        : integer;
1071        signal threshold_red1         : integer;
1072        signal threshold_red2         : integer;
1073        signal threshold_gre1         : integer;
1074        signal threshold_gre2         : integer;
1075        signal threshold_blu1         : integer;
1076        signal threshold_blu2         : integer;
1077        signal threshold_lms          : integer;
1078        signal rgb_min1               : integer;
1079        signal rgb_max1               : integer;
1080        signal rgb_max2               : integer;
1081        signal rgb_max3               : integer;
1082        signal rgb_max4               : integer;
1083        signal rgb_max5               : integer;
1084        signal rgb_max6               : integer;
1085        signal rgb_max7               : integer;
1086        signal rgb_max8               : integer;
1087        signal rgb_max9               : integer;
1088        signal rgb_max10              : integer;
1089        signal rgb_max11              : integer;
1090        signal rgb_max12              : integer;
1091        signal rgb_max13              : integer;
1092        signal rgb_max14              : integer;
1093        signal rgb_max15              : integer;
1094        signal rgb_max16              : integer;
1095        signal rgb_max17              : integer;
1096        signal rgb_max18              : integer;
1097        signal rgb_max19              : integer;
1098        signal rgb_max20              : integer;
1099        signal rgb_max21              : integer;
1100        signal rgb_max22              : integer;
1101        signal rgb_std1sync           : channel;
1102        signal rgb_std2sync           : channel;
1103        signal rgb_std3sync           : channel;
1104        signal rgb_std4sync           : channel;
1105        signal rgb_sync1              : intChannel;
1106        signal rgb_sync2              : intChannel;
1107        signal rgb_sync3              : intChannel;
1108        signal rgb_sync4              : intChannel;
1109        signal rgb_sync5              : intChannel;
1110        signal rgb_sync6              : intChannel;
```

```
1111        signal rgb_sync7              : intChannel;
1112        signal rgb_sync8              : intChannel;
1113        signal rgb_sync9              : intChannel;
1114        signal rgb_sync10             : intChannel;
1115        signal rgb_sync11             : intChannel;
1116        signal rgb_sync12             : intChannel;
1117        signal rgb_sync13             : intChannel;
1118        signal rgb_sync14             : intChannel;
1119        signal rgb_sync15             : intChannel;
1120        signal rgb_sync16             : intChannel;
1121        signal rgb_sync17             : intChannel;
1122        signal rgb_sync18             : intChannel;
1123        signal rgb_sync19             : intChannel;
1124        signal rgb_sync20             : intChannel;
1125        signal rgb_sync21             : intChannel;
1126        signal rgb_sync22             : intChannel;
1127        signal rgb_sync23             : intChannel;
1128        signal rgb_max                : integer;
1129        signal rgb_mid                : integer;
1130        signal rgb_min                : integer;
1131        signal rgb_red                : std_logic_vector(7 downto 0);
1132        signal rgb_gre                : std_logic_vector(7 downto 0);
1133        signal rgb_blu                : std_logic_vector(7 downto 0);
1134        constant K_VALUE              : integer := 11;
1135        attribute keep : string;
1136        attribute keep of rgb_max : signal is "true";
1137        attribute keep of rgb_mid : signal is "true";
1138        attribute keep of rgb_min : signal is "true";
1139
1140    begin
1141
1142    process (clk) begin
1143        if rising_edge(clk) then
1144            rgbSyncValid(0)  <= pixel_in_rgb.valid;
1145            for i in 0 to 30 loop
1146              rgbSyncValid(i+1)  <= rgbSyncValid(i);
1147            end loop;
1148        end if;
1149    end process;
1150
1151    process (clk) begin
1152        if rising_edge(clk) then
1153            rgbSyncEol(0)  <= pixel_in_rgb.eol;
1154            for i in 0 to 30 loop
1155              rgbSyncEol(i+1)  <= rgbSyncEol(i);
1156            end loop;
1157        end if;
1158    end process;
1159
1160    process (clk) begin
1161        if rising_edge(clk) then
1162            rgbSyncSof(0)  <= pixel_in_rgb.sof;
1163            for i in 0 to 30 loop
1164              rgbSyncSof(i+1)  <= rgbSyncSof(i);
1165            end loop;
1166        end if;
1167    end process;
1168
1169    process (clk) begin
1170        if rising_edge(clk) then
1171            rgbSyncEof(0)  <= pixel_in_rgb.eof;
1172            for i in 0 to 30 loop
1173              rgbSyncEof(i+1)  <= rgbSyncEof(i);
1174            end loop;
1175        end if;
1176    end process;
1177
1178    process (clk)begin
1179        if rising_edge(clk) then
1180            rgb_std1sync.red    <= pixel_in_rgb.red;
1181            rgb_std1sync.green  <= pixel_in_rgb.green;
1182            rgb_std1sync.blue   <= pixel_in_rgb.blue;
1183            rgb_std1sync.valid  <= pixel_in_rgb.valid;
1184        end if;
```

```
1185  end process;
1186
1187  process (clk)begin
1188      if rising_edge(clk) then
1189          rgb_std2sync    <= rgb_std1sync;
1190          rgb_std3sync    <= rgb_std2sync;
1191          rgb_std4sync    <= rgb_std3sync;
1192      end if;
1193  end process;
1194
1195  process (clk)begin
1196      if rising_edge(clk) then
1197              rgb_sync1.red    <= to_integer(unsigned(rgb_std1sync.red(9 downto 2)));
1198              rgb_sync1.green  <= to_integer(unsigned(rgb_std1sync.green(9 downto 2)));
1199              rgb_sync1.blue   <= to_integer(unsigned(rgb_std1sync.blue(9 downto 2)));
1200      end if;
1201  end process;
1202
1203  process (clk) begin
1204      if rising_edge(clk) then
1205          if ((rgb_sync1.red >= rgb_sync1.green) and (rgb_sync1.red >= rgb_sync1.blue)) then
1206              rgb_max1 <= rgb_sync1.red;
1207          elsif ((rgb_sync1.green >= rgb_sync1.red) and (rgb_sync1.green >= rgb_sync1.blue)) then
1208              rgb_max1 <= rgb_sync1.green;
1209          else
1210              rgb_max1 <= rgb_sync1.blue;
1211          end if;
1212      end if;
1213  end process;
1214
1215  process (clk) begin
1216      if rising_edge(clk) then
1217          if ((rgb_sync1.red <= rgb_sync1.green) and (rgb_sync1.red <= rgb_sync1.blue)) then
1218              rgb_min1 <= rgb_sync1.red;
1219          elsif((rgb_sync1.green <= rgb_sync1.red) and (rgb_sync1.green <= rgb_sync1.blue)) then
1220              rgb_min1 <= rgb_sync1.green;
1221          else
1222              rgb_min1 <= rgb_sync1.blue;
1223          end if;
1224      end if;
1225  end process;
1226
1227  process (clk) begin
1228      if rising_edge(clk) then
1229          if (rgb_sync2.red = rgb_max1) and (rgb_sync2.green = rgb_min1) then
1230              rgb_max <= rgb_sync2.red;
1231              rgb_mid <= rgb_sync2.blue;
1232              rgb_min <= rgb_sync2.green;
1233          elsif(rgb_sync2.red = rgb_max1) and (rgb_sync2.blue = rgb_min1)then
1234              rgb_max <= rgb_sync2.red;
1235              rgb_mid <= rgb_sync2.green;
1236              rgb_min <= rgb_sync2.blue;
1237          elsif(rgb_sync2.green = rgb_max1) and (rgb_sync2.blue = rgb_min1)then
1238              rgb_max <= rgb_sync2.green;
1239              rgb_mid <= rgb_sync2.red;
1240              rgb_min <= rgb_sync2.blue;
1241          elsif(rgb_sync2.green = rgb_max1) and (rgb_sync2.red = rgb_min1)then
1242              rgb_max <= rgb_sync2.green;
1243              rgb_mid <= rgb_sync2.blue;
1244              rgb_min <= rgb_sync2.red;
1245          elsif(rgb_sync2.blue = rgb_max1) and (rgb_sync2.red = rgb_min1)then
1246              rgb_max <= rgb_sync2.blue;
1247              rgb_mid <= rgb_sync2.green;
1248              rgb_min <= rgb_sync2.red;
1249          elsif(rgb_sync2.blue = rgb_max1) and (rgb_sync2.green = rgb_min1)then
1250              rgb_max <= rgb_sync2.blue;
1251              rgb_mid <= rgb_sync2.red;
1252              rgb_min <= rgb_sync2.green;
1253          end if;
1254      end if;
1255  end process;
1256
1257  process (clk)begin
1258      if rising_edge(clk) then
```

```
1259        rgb_max2        <= rgb_max1;
1260        rgb_max3        <= rgb_max2;
1261        rgb_max4        <= rgb_max3;
1262        rgb_max5        <= rgb_max4;
1263        rgb_max6        <= rgb_max5;
1264        rgb_max7        <= rgb_max6;
1265        rgb_max8        <= rgb_max7;
1266        rgb_max9        <= rgb_max8;
1267        rgb_max10       <= rgb_max9;
1268        rgb_max11       <= rgb_max10;
1269        rgb_max12       <= rgb_max11;
1270        rgb_max13       <= rgb_max12;
1271        rgb_max14       <= rgb_max13;
1272        rgb_max15       <= rgb_max14;
1273        rgb_max16       <= rgb_max15;
1274        rgb_max17       <= rgb_max16;
1275        rgb_max18       <= rgb_max17;
1276        rgb_max19       <= rgb_max18;
1277        rgb_max20       <= rgb_max19;
1278        rgb_max21       <= rgb_max20;
1279        rgb_max22       <= rgb_max21;
1280      end if;
1281  end process;
1282  process (clk)begin
1283      if rising_edge(clk) then
1284        rgb_sync2        <= rgb_sync1;
1285        rgb_sync3        <= rgb_sync2;
1286        rgb_sync4        <= rgb_sync3;
1287        rgb_sync5        <= rgb_sync4;
1288        rgb_sync6        <= rgb_sync5;
1289        rgb_sync7        <= rgb_sync6;
1290        rgb_sync8        <= rgb_sync7;
1291        rgb_sync9        <= rgb_sync8;
1292        rgb_sync10       <= rgb_sync9;
1293        rgb_sync11       <= rgb_sync10;
1294        rgb_sync12       <= rgb_sync11;
1295        rgb_sync13       <= rgb_sync12;
1296        rgb_sync14       <= rgb_sync13;
1297        rgb_sync15       <= rgb_sync14;
1298        rgb_sync16       <= rgb_sync15;
1299        rgb_sync17       <= rgb_sync16;
1300        rgb_sync18       <= rgb_sync17;
1301        rgb_sync19       <= rgb_sync18;
1302        rgb_sync20       <= rgb_sync19;
1303        rgb_sync21       <= rgb_sync20;
1304        rgb_sync22       <= rgb_sync21;
1305        rgb_sync23       <= rgb_sync22;
1306      end if;
1307  end process;
1308  -- best select is 18/17/16
1309  process (clk)begin
1310      if rising_edge(clk) then
1311          if (k_ind_w <= 30)then
1312              k_rgb_lut_4ll(k_ind_w).max  <= to_integer((unsigned(centroid_lut_in(23 downto 16))));
                   k_rgb_lut_4ll(k_ind_w).mid <= to_integer((unsigned(centroid_lut_in(15 downto 8))));
                     k_rgb_lut_4ll(k_ind_w).min <= to_integer((unsigned(centroid_lut_in(7 downto 0))));
1313          end if;
1314      end if;
1315  end process;
1316
1317  process (clk)begin
1318      if rising_edge(clk) then
1319          if (k_ind_w >= 31 and k_ind_w <= 60) then
1320              k_rgb_lut_2ll(61-k_ind_w).max  <= to_integer((unsigned(centroid_lut_in(23 downto 16
                  )))); k_rgb_lut_2ll(61-k_ind_w).mid <= to_integer((unsigned(centroid_lut_in(15 downto
                   8))));   k_rgb_lut_2ll(61-k_ind_w).min <= to_integer((unsigned(centroid_lut_in(7
                  downto 0))));
1321          end if;
1322      end if;
1323  end process;
1324
1325  process (clk)begin
1326      if rising_edge(clk) then
1327          if (k_ind_w >= 61 and k_ind_w <= 90) then
```

```
            k_rgb_lut_3ll(91-k_ind_w).max  <= to_integer((unsigned(centroid_lut_in(23 downto 16 )))); k_rgb_lut_3ll(91-k_ind_w).mid <= to_integer((unsigned(centroid_lut_in(15 downto 8)))); k_rgb_lut_3ll(91-k_ind_w).min <= to_integer((unsigned(centroid_lut_in(7 downto 0))));
        end if;
    end if;
end process;

process (clk)begin
    if rising_edge(clk) then
        if (k_ind_w >= 91 and k_ind_w <= 120) then
            k_rgb_lut_6ll(121-k_ind_w).max  <= to_integer((unsigned(centroid_lut_in(23 downto 16 )))); k_rgb_lut_6ll(121-k_ind_w).mid <= to_integer((unsigned(centroid_lut_in(15 downto 8)))); k_rgb_lut_6ll(121-k_ind_w).min <= to_integer((unsigned(centroid_lut_in(7 downto 0))));
        end if;
    end if;
end process;

process(clk)begin
    if rising_edge(clk) then
        if (k_ind_r <= 30)then
            centroid_lut_out                <= x"00" & std_logic_vector(to_unsigned(k_rgb_lut_4_l(k_ind_r).max, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_4_l(k_ind_r).mid, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_4_l(k_ind_r).min, 8));
        elsif(k_ind_w >= 31 and k_ind_w <= 60)then
            centroid_lut_out                <= x"00" & std_logic_vector(to_unsigned(k_rgb_lut_2_l(61-k_ind_r).max, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_2_l(61-k_ind_r).mid, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_2_l(61-k_ind_r).min, 8));
        elsif(k_ind_w >= 61 and k_ind_w <= 90)then
            centroid_lut_out                <= x"00" & std_logic_vector(to_unsigned(k_rgb_lut_3_l(91-k_ind_r).max, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_3_l(91-k_ind_r).mid, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_3_l(91-k_ind_r).min, 8));
        elsif(k_ind_w >= 91 and k_ind_w <= 120)then
            centroid_lut_out                <= x"00" & std_logic_vector(to_unsigned(k_rgb_lut_6_l(121-k_ind_r).max, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_6_l(121-k_ind_r).mid, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_6_l(121-k_ind_r).min, 8));
        else
            centroid_lut_out                <= x"00" & std_logic_vector(to_unsigned(k_rgb_lut_3_l(91-k_ind_r).max, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_3_l(91-k_ind_r).mid, 8)) & std_logic_vector(to_unsigned(k_rgb_lut_3_l(91-k_ind_r).min, 8));
        end if;
    end if;
end process;



process (clk)begin
    if rising_edge(clk) then
        if (k_ind_w <= 90)then
            k_rgb_lut_4_l <= k_rgb_lut_4ll;
            k_rgb_lut_3_l <= k_rgb_lut_3ll;
            k_rgb_lut_2_l <= k_rgb_lut_2ll;
            k_rgb_lut_6_l <= k_rgb_lut_6ll;
        elsif(k_ind_w = 201)then
            k_rgb_lut_4_l <= k_rgb_lut_42l;
            k_rgb_lut_3_l <= k_rgb_lut_3ll;
            k_rgb_lut_2_l <= k_rgb_lut_22l;
            k_rgb_lut_6_l <= k_rgb_lut_6ll;
        elsif(k_ind_w = 202)then
            k_rgb_lut_4_l <= k_rgb_lut_43l;
            k_rgb_lut_3_l <= k_rgb_lut_3ll;
            k_rgb_lut_2_l <= k_rgb_lut_22l;
            k_rgb_lut_6_l <= k_rgb_lut_6ll;
        elsif(k_ind_w = 203)then
            k_rgb_lut_4_l <= k_rgb_lut_41l;
            k_rgb_lut_3_l <= k_rgb_lut_3ll;
            k_rgb_lut_2_l <= k_rgb_lut_22l;
            k_rgb_lut_6_l <= k_rgb_lut_6ll;
        elsif(k_ind_w = 204)then
            k_rgb_lut_4_l <= k_rgb_lut_42l;
            k_rgb_lut_3_l <= k_rgb_lut_3ll;
            k_rgb_lut_2_l <= k_rgb_lut_22l;
            k_rgb_lut_6_l <= k_rgb_lut_6ll;
```

```
1386            elsif(k_ind_w = 205)then
1387                k_rgb_lut_4_l <= k_rgb_lut_43l;
1388                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1389                k_rgb_lut_2_l <= k_rgb_lut_22l;
1390                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1391            elsif(k_ind_w = 206)then
1392                k_rgb_lut_4_l <= k_rgb_lut_44l;
1393                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1394                k_rgb_lut_2_l <= k_rgb_lut_22l;
1395                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1396            elsif(k_ind_w = 207)then
1397                k_rgb_lut_4_l <= k_rgb_lut_45l;
1398                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1399                k_rgb_lut_2_l <= k_rgb_lut_22l;
1400                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1401            elsif(k_ind_w = 208)then
1402                k_rgb_lut_4_l <= k_rgb_lut_46l;
1403                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1404                k_rgb_lut_2_l <= k_rgb_lut_22l;
1405                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1406            elsif(k_ind_w = 209)then
1407                k_rgb_lut_4_l <= k_rgb_lut_47l;
1408                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1409                k_rgb_lut_2_l <= k_rgb_lut_22l;
1410                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1411            elsif(k_ind_w =210)then
1412                k_rgb_lut_4_l <= k_rgb_lut_48l;
1413                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1414                k_rgb_lut_2_l <= k_rgb_lut_22l;
1415                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1416            elsif(k_ind_w =211)then
1417                k_rgb_lut_4_l <= k_rgb_lut_49l;
1418                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1419                k_rgb_lut_2_l <= k_rgb_lut_22l;
1420                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1421            elsif(k_ind_w =212)then
1422                k_rgb_lut_4_l <= k_rgb_lut_50l;
1423                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1424                k_rgb_lut_2_l <= k_rgb_lut_22l;
1425                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1426            elsif(k_ind_w =213)then
1427                k_rgb_lut_4_l <= k_rgb_lut_51l;
1428                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1429                k_rgb_lut_2_l <= k_rgb_lut_22l;
1430                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1431            elsif(k_ind_w =214)then
1432                k_rgb_lut_4_l <= k_rgb_lut_52l;
1433                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1434                k_rgb_lut_2_l <= k_rgb_lut_22l;
1435                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1436            elsif(k_ind_w =215)then
1437                k_rgb_lut_4_l <= k_rgb_lut_53l;
1438                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1439                k_rgb_lut_2_l <= k_rgb_lut_22l;
1440                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1441            elsif(k_ind_w =216)then
1442                k_rgb_lut_4_l <= k_rgb_lut_54l;
1443                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1444                k_rgb_lut_2_l <= k_rgb_lut_22l;
1445                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1446            elsif(k_ind_w =217)then
1447                k_rgb_lut_4_l <= k_rgb_lut_55l;
1448                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1449                k_rgb_lut_2_l <= k_rgb_lut_22l;
1450                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1451            elsif(k_ind_w =218)then
1452                k_rgb_lut_4_l <= k_rgb_lut_56l;
1453                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1454                k_rgb_lut_2_l <= k_rgb_lut_22l;
1455                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1456            elsif(k_ind_w =219)then
1457                k_rgb_lut_4_l <= k_rgb_lut_57l;
1458                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1459                k_rgb_lut_2_l <= k_rgb_lut_22l;
```

```
1460                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1461            elsif(k_ind_w =220)then
1462                k_rgb_lut_4_l <= k_rgb_lut_58l;
1463                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1464                k_rgb_lut_2_l <= k_rgb_lut_22l;
1465                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1466            elsif(k_ind_w =221)then
1467                k_rgb_lut_4_l <= k_rgb_lut_59l;
1468                k_rgb_lut_3_l <= k_rgb_lut_3ll;
1469                k_rgb_lut_2_l <= k_rgb_lut_22l;
1470                k_rgb_lut_6_l <= k_rgb_lut_6ll;
1471            end if;
1472        end if;
1473    end process;
1474    -- best select is 0
1475    process (clk)begin
1476        if rising_edge(clk) then
1477            if (centroid_lut_select   = 0)then
1478
                ------------------
```

```
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_1_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_1_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_1_l(i).min;
                          end loop;
                    end if;
                elsif(rgb_sync2.green >= 100) then
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
                          end loop;
                    end if;
                else
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
                          end loop;
                    end if;
                end if;
            else
                if (rgb_max1 - rgb_min1 >= 100) then
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_1_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_1_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_1_l(i).max;
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_1_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_1_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_1_l(i).max;
                          end loop;
                    end if;
                elsif(rgb_sync2.blue >= 100) then
                        if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
                          end loop;
                        else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
                          end loop;
                        end if;
                else
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
                          end loop;
                    end if;
                end if;
            end if;

            ------------------------------------------------------------------------------------
            --------------------
        elsif(centroid_lut_select = 1) then
```

```
1582
                ------------------------------------------------------------------------------------------------------------------
1583            if ((abs(rgb_sync2.red - rgb_sync2.green) <= 6) and (abs(rgb_sync2.red - rgb_sync2.blue) <= 6)) or (abs(rgb_sync2.red - rgb_sync2.blue) <= 6 and (abs(rgb_sync2.red - rgb_sync2.green) <= 6)) or (abs(rgb_sync2.green - rgb_sync2.blue) <= 6  and (abs(rgb_sync2.red - rgb_sync2.blue) <= 6))then
1584                for i in 0 to 30 loop
1585                    k_rgb(i).red <=   k_rgb_lut_0_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_0_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_0_l(i).mid;
1586                end loop;
1587            -- RED MAX
1588            elsif (rgb_sync2.red = rgb_max1) then
1589                if (rgb_sync2.red >= 100) then
1590                    if (rgb_sync2.green = rgb_min1) then
1591                        for i in 0 to 30 loop
1592                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
1593                        end loop;
1594                    else
1595                        for i in 0 to 30 loop
1596                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
1597                        end loop;
1598                    end if;
1599                else
1600                    if (rgb_sync2.green = rgb_min1) then
1601                        for i in 0 to 30 loop
1602                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
1603                        end loop;
1604                    else
1605                        for i in 0 to 30 loop
1606                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
1607                        end loop;
1608                    end if;
1609                end if;
1610            -- GREEN MAX
1611            elsif (rgb_sync2.green = rgb_max1) then
1612                if(rgb_sync2.green >= 100) then
1613                    if (rgb_sync2.red = rgb_min1) then
1614                          for i in 0 to 30 loop
1615                            k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=  k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
1616                          end loop;
1617                    else
1618                          for i in 0 to 30 loop
1619                            k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=  k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
1620                          end loop;
1621                    end if;
1622                else
1623                    if (rgb_sync2.red = rgb_min1) then
1624                          for i in 0 to 30 loop
1625                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=  k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
1626                          end loop;
1627                    else
1628                          for i in 0 to 30 loop
1629                            k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=  k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
1630                          end loop;
1631                    end if;
1632                end if;
1633            else
1634            -- BLUE MAX
1635                if(rgb_sync2.blue >= 100) then
1636                    if (rgb_sync2.red = rgb_min1) then
1637                        for i in 0 to 30 loop
1638                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
1639                        end loop;
1640                    else
```

```
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
                        end loop;
                    end if;
                else
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
                          end loop;
                    end if;
                end if;
            end if;

            ------------------------------------------------------------------------------------
            --------------------
        elsif(centroid_lut_select = 2) then

            ------------------------------------------------------------------------------------
            --------------------
            if (rgb_sync2.red = rgb_max1) then
                if (rgb_sync2.red >= 100) then
                    if (rgb_sync2.green = rgb_min1) then
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
                        end loop;
                    else
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
                        end loop;
                    end if;
                else
                    if (rgb_sync2.green = rgb_min1) then
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
                        end loop;
                    else
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
                        end loop;
                    end if;
                end if;
            -- GREEN MAX
            elsif (rgb_sync2.green = rgb_max1) then
                if(rgb_sync2.green >= 100) then
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
                          end loop;
                    end if;
                else
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
                          end loop;
                    else
                          for i in 0 to 30 loop
```

```
1701                            k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
1702                          end loop;
1703                      end if;
1704                  end if;
1705              else
1706              -- BLUE MAX
1707                  if(rgb_sync2.blue >= 100) then
1708                      if (rgb_sync2.red = rgb_min1) then
1709                          for i in 0 to 30 loop
1710                              k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                  k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
1711                          end loop;
1712                      else
1713                          for i in 0 to 30 loop
1714                              k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=
                                  k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
1715                          end loop;
1716                      end if;
1717                  else
1718                      if (rgb_sync2.red = rgb_min1) then
1719                            for i in 0 to 30 loop
1720                              k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                  k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
1721                            end loop;
1722                      else
1723                            for i in 0 to 30 loop
1724                              k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=
                                  k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
1725                            end loop;
1726                      end if;
1727                  end if;
1728              end if;
1729
                  ------------------------------------------------------------------------------------------
                  --------------------
1730          elsif(centroid_lut_select = 3) then
1731
                  ------------------------------------------------------------------------------------------
                  --------------------
1732              if (rgb_sync2.red = rgb_max1) then
1733                  if (rgb_sync2.red >= 100) then
1734                      if (rgb_sync2.green = rgb_min1) then
1735                          for i in 0 to 30 loop
1736                              k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                  k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
1737                          end loop;
1738                      else
1739                          for i in 0 to 30 loop
1740                              k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                  k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
1741                          end loop;
1742                      end if;
1743                  else
1744                      if (rgb_sync2.green = rgb_min1) then
1745                          for i in 0 to 30 loop
1746                              k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                  k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
1747                          end loop;
1748                      else
1749                          for i in 0 to 30 loop
1750                              k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                  k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
1751                          end loop;
1752                      end if;
1753                  end if;
1754              -- GREEN MAX
1755              elsif (rgb_sync2.green = rgb_max1) then
1756                  if(rgb_sync2.green >= 100) then
1757                      if (rgb_sync2.red = rgb_min1) then
1758                            for i in 0 to 30 loop
1759                              k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                  k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
1760                            end loop;
```

```
1761                    else
1762                          for i in 0 to 30 loop
1763                            k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
1764                          end loop;
1765                    end if;
1766                else
1767                    if (rgb_sync2.red = rgb_min1) then
1768                          for i in 0 to 30 loop
1769                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
1770                          end loop;
1771                    else
1772                          for i in 0 to 30 loop
1773                            k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
1774                          end loop;
1775                    end if;
1776                end if;
1777            else
1778            -- BLUE MAX
1779                if(rgb_sync2.blue >= 100) then
1780                    if (rgb_sync2.red = rgb_min1) then
1781                        for i in 0 to 30 loop
1782                            k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
1783                        end loop;
1784                    else
1785                        for i in 0 to 30 loop
1786                            k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
1787                        end loop;
1788                    end if;
1789                else
1790                    if (rgb_sync2.red = rgb_min1) then
1791                          for i in 0 to 30 loop
1792                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
1793                          end loop;
1794                    else
1795                          for i in 0 to 30 loop
1796                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
1797                          end loop;
1798                    end if;
1799                end if;
1800            end if;
1801
                ------------------------------------------------------------------------------------
                --------------------
1802        elsif(centroid_lut_select = 4) then
1803
                ------------------------------------------------------------------------------------
                --------------------
1804            if (rgb_sync2.red = rgb_max1) then
1805                if (rgb_sync2.red >= 100) then
1806                    if (rgb_sync2.green = rgb_min1) then
1807                        for i in 0 to 30 loop
1808                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
1809                        end loop;
1810                    else
1811                        for i in 0 to 30 loop
1812                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
1813                        end loop;
1814                    end if;
1815                else
1816                    if (rgb_sync2.green = rgb_min1) then
1817                        for i in 0 to 30 loop
1818                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
1819                        end loop;
1820                    else
```

```
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
                        end loop;
                    end if;
                end if;
            -- GREEN MAX
            elsif (rgb_sync2.green = rgb_max1) then
                if(rgb_sync2.green >= 100) then
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
                          end loop;
                    end if;
                else
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
                          end loop;
                    end if;
                end if;
            else
            -- BLUE MAX
                if(rgb_sync2.blue >= 100) then
                    if (rgb_sync2.red = rgb_min1) then
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
                        end loop;
                    else
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
                        end loop;
                    end if;
                else
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
                          end loop;
                    else
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).mid;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
                          end loop;
                    end if;
                end if;
            end if;

            ------------------
```

```
1880                        for i in 0 to 30 loop
1881                            k_rgb(i).red <=   k_rgb_lut_6_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_6_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_6_l(i).mid;
1882                        end loop;
1883                    else
1884                        for i in 0 to 30 loop
1885                            k_rgb(i).red <=   k_rgb_lut_6_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_6_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_6_l(i).mid;
1886                        end loop;
1887                    end if;
1888                elsif(rgb_sync2.red >= 170) then
1889
                        ------------------------------------------------------------------------
                        --------------------
1890                    if (rgb_sync2.green = rgb_min1) then
1891                        for i in 0 to 30 loop
1892                            k_rgb(i).red <=   k_rgb_lut_3_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_3_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
1893                        end loop;
1894                    else
1895                        for i in 0 to 30 loop
1896                            k_rgb(i).red <=   k_rgb_lut_3_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_3_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
1897                        end loop;
1898                    end if;
1899
                        ------------------------------------------------------------------------
                        --------------------
1900                elsif(rgb_sync2.red >= 85) then
1901
                        ------------------------------------------------------------------------
                        --------------------
1902                    if (rgb_sync2.green = rgb_min1) then
1903                        for i in 0 to 30 loop
1904                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
1905                        end loop;
1906                    else
1907                        for i in 0 to 30 loop
1908                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
1909                        end loop;
1910                    end if;
1911
                        ------------------------------------------------------------------------
                        --------------------
1912                else
1913
                        ------------------------------------------------------------------------
                        --------------------
1914                    if (rgb_sync2.green = rgb_min1) then
1915                        for i in 0 to 30 loop
1916                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
1917                        end loop;
1918                    else
1919                        for i in 0 to 30 loop
1920                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
1921                        end loop;
1922                    end if;
1923
                        ------------------------------------------------------------------------
                        --------------------
1924                end if;
1925            -- GREEN MAX
1926            elsif (rgb_sync2.green = rgb_max1) then
1927                if(rgb_sync2.green >= 210) then
1928                    if (rgb_sync2.red = rgb_min1) then
1929                          for i in 0 to 30 loop
1930                            k_rgb(i).red <=   k_rgb_lut_6_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_6_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_6_l(i).mid;
1931                          end loop;
1932                    else
```

```
                        for i in 0 to 30 loop
                          k_rgb(i).red <=   k_rgb_lut_6_l(i).min;   k_rgb(i).gre <=
                          k_rgb_lut_6_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_6_l(i).mid;
                        end loop;
                    end if;
                elsif(rgb_sync2.green >= 170) then
                    if (rgb_sync2.red = rgb_min1) then
                        for i in 0 to 30 loop
                          k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                          k_rgb_lut_3_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
                        end loop;
                    else
                        for i in 0 to 30 loop
                          k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                          k_rgb_lut_3_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
                        end loop;
                    end if;

                    ------------------------------------------------------------------------------------------------------------------------
                elsif(rgb_sync2.red >= 85) then

                    ------------------------------------------------------------------------------------------------------------------------
                    if (rgb_sync2.red = rgb_min1) then
                        for i in 0 to 30 loop
                          k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                          k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
                        end loop;
                    else
                        for i in 0 to 30 loop
                          k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                          k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
                        end loop;
                    end if;

                    ------------------------------------------------------------------------------------------------------------------------
                else

                    ------------------------------------------------------------------------------------------------------------------------
                    if (rgb_sync2.red = rgb_min1) then
                        for i in 0 to 30 loop
                          k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                          k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
                        end loop;
                    else
                        for i in 0 to 30 loop
                          k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                          k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
                        end loop;
                    end if;
                end if;
            else
            -- BLUE MAX
                if(rgb_sync2.blue >= 210) then
                    if (rgb_sync2.red = rgb_min1) then
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_6_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_6_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_6_l(i).max;
                        end loop;
                    else
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_6_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_6_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_6_l(i).max;
                        end loop;
                    end if;
                elsif(rgb_sync2.blue >= 170) then
                    if (rgb_sync2.red = rgb_min1) then
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).max;
                        end loop;
```

```
                    else
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).max;
                        end loop;
                    end if;

                    ------------------
```

```
                    if (rgb_sync2.green = rgb_min1) then
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
                        end loop;
                    else
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).min;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
                        end loop;
                    end if;

                    ------------------
```

```
2090                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
2091                          end loop;
2092                      else
2093                          for i in 0 to 30 loop
2094                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
2095                          end loop;
2096                      end if;
2097                  end if;
2098              else
2099
                  ------------------------------------------------------------------------------------
                  --------------------
2100              -- BLUE MAX
2101                  if(rgb_sync2.blue >= 170) then
2102                      if (rgb_sync2.red = rgb_min1) then
2103                          for i in 0 to 30 loop
2104                              k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                                  k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).max;
2105                          end loop;
2106                      else
2107                          if ((abs(rgb_max - rgb_mid) <= 20) or (abs(rgb_max - rgb_min) <= 20))then
2108                              for i in 0 to 30 loop
2109                                  k_rgb(i).red <=   k_rgb_lut_3_l(i).max;   k_rgb(i).gre <=
                                      k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).min;
2110                              end loop;
2111                          else
2112                              for i in 0 to 30 loop
2113                                  k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                                      k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).max;
2114                              end loop;
2115                          end if;
2116                      end if;
2117
                          ------------------------------------------------------------------------------
                          --------------------
2118                  elsif(rgb_sync2.red >= 85) then
2119
                          ------------------------------------------------------------------------------
                          --------------------
2120                      if (rgb_sync2.red = rgb_min1) then
2121                          for i in 0 to 30 loop
2122                              k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                  k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
2123                          end loop;
2124                      else
2125                          if ((abs(rgb_max - rgb_mid) <= 40) or (abs(rgb_max - rgb_min) <= 40))then
2126                              for i in 0 to 30 loop
2127                                  k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                      k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
2128                              end loop;
2129                          else
2130                              for i in 0 to 30 loop
2131                                  k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                      k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
2132                              end loop;
2133                          end if;
2134                      end if;
2135
                          ------------------------------------------------------------------------------
                          --------------------
2136                  else
2137
                          ------------------------------------------------------------------------------
                          --------------------
2138                      if (rgb_sync2.red = rgb_min1) then
2139                            for i in 0 to 30 loop
2140                              k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                  k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
2141                            end loop;
2142                      else
2143                          if ((abs(rgb_max - rgb_mid) <= 8) or (abs(rgb_max - rgb_min) <= 8))then
2144                              for i in 0 to 30 loop
```

```
                                k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=  abs(k_rgb_lut_2_l(i).mid - 30);   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
                            end loop;
                        else
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=  k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
                            end loop;
                        end if;
                    end if;
                end if;
            end if;

            ------------------------------------------------------------------------------------------------------------------------------------------------------
        elsif(centroid_lut_select = 7) then

            ------------------------------------------------------------------------------------------------------------------------------------------------------
            -- RED MAX
            if (rgb_sync2.red = rgb_max1) then
                if (rgb_sync2.red >= 170) then

                    ------------------------------------------------------------------------------------------------------------------------------------------------------
                    if (rgb_sync2.green = rgb_min1) then
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_3_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).min;
                        end loop;
                    else
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_3_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).min;
                        end loop;
                    end if;

                    ------------------------------------------------------------------------------------------------------------------------------------------------------
                elsif(rgb_sync2.red >= 85) then

                    ------------------------------------------------------------------------------------------------------------------------------------------------------
                    if (rgb_sync2.green = rgb_min1) then
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
                        end loop;
                    else
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
                        end loop;
                    end if;

                    ------------------------------------------------------------------------------------------------------------------------------------------------------
                else

                    ------------------------------------------------------------------------------------------------------------------------------------------------------
                    if (rgb_sync2.green = rgb_min1) then
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
                        end loop;
                    else
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).min;
                        end loop;
                    end if;

```

```
                ------------------------------------------------------------------------------------------------------------------
            end if;

        ------------------------------------------------------------------------------------------------------------------
        -- GREEN MAX
        elsif (rgb_sync2.green = rgb_max1) then
            if(rgb_sync2.green >= 170) then
                if (rgb_sync2.red = rgb_min1) then
                      for i in 0 to 30 loop
                        k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=   k_rgb_lut_3_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
                      end loop;
                else
                      for i in 0 to 30 loop
                        k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=   k_rgb_lut_3_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).mid;
                      end loop;
                end if;

                ------------------------------------------------------------------------------------------------------------------
            elsif(rgb_sync2.red >= 85) then

                ------------------------------------------------------------------------------------------------------------------
                if (rgb_sync2.red = rgb_min1) then
                      for i in 0 to 30 loop
                        k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=   k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
                      end loop;
                else
                      for i in 0 to 30 loop
                        k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=   k_rgb_lut_4_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).mid;
                      end loop;
                end if;

                ------------------------------------------------------------------------------------------------------------------
            else

                ------------------------------------------------------------------------------------------------------------------
                if (rgb_sync2.red = rgb_min1) then
                      for i in 0 to 30 loop
                        k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=   k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
                      end loop;
                else
                      for i in 0 to 30 loop
                        k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=   k_rgb_lut_2_l(i).max;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).mid;
                      end loop;
                end if;
            end if;
        else

        ------------------------------------------------------------------------------------------------------------------
        -- BLUE MAX
            if(rgb_sync2.blue >= 170) then
                if (rgb_sync2.red = rgb_min1) then
                    for i in 0 to 30 loop
                        k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=   k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).max;
                    end loop;
                else
                    if ((abs(rgb_max - rgb_mid) <= 20) or (abs(rgb_max - rgb_min) <= 20))then
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_3_l(i).max;   k_rgb(i).gre <=   k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).min;
                        end loop;
```

```
                        else
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_3_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_3_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_3_l(i).max;
                            end loop;
                        end if;
                    end if;

                    ------------------------------------------------------------------------------
                    --------------------
                elsif(rgb_sync2.red >= 85) then

                    ------------------------------------------------------------------------------
                    --------------------
                    if (rgb_sync2.red = rgb_min1) then
                        for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
                        end loop;
                    else
                        if ((abs(rgb_max - rgb_mid) <= 40) or (abs(rgb_max - rgb_min) <= 40))then
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_4_l(i).max;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).min;
                            end loop;
                        else
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_4_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_4_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_4_l(i).max;
                            end loop;
                        end if;
                    end if;

                    ------------------------------------------------------------------------------
                    --------------------
                else

                    ------------------------------------------------------------------------------
                    --------------------
                    if (rgb_sync2.red = rgb_min1) then
                          for i in 0 to 30 loop
                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                            k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
                          end loop;
                    else
                        if ((abs(rgb_max - rgb_mid) <= 8) or (abs(rgb_max - rgb_min) <= 8))then
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_2_l(i).max;   k_rgb(i).gre <=  abs(
                                k_rgb_lut_2_l(i).mid - 30);   k_rgb(i).blu <=  k_rgb_lut_2_l(i).
                                min;
                            end loop;
                        else
                            for i in 0 to 30 loop
                                k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
                            end loop;
                        end if;
                    end if;
                end if;
            end if;

            ------------------------------------------------------------------------------------
            --------------------
        elsif(centroid_lut_select = 9) then

            ------------------------------------------------------------------------------------
            --------------------
            if ((abs(rgb_sync2.red - rgb_sync2.green) <= 6) and (abs(rgb_sync2.red - rgb_sync2.
            blue) <= 6)) or (abs(rgb_sync2.red - rgb_sync2.blue) <= 6 and (abs(rgb_sync2.red -
            rgb_sync2.green) <= 6)) or (abs(rgb_sync2.green - rgb_sync2.blue) <= 6  and (abs(
            rgb_sync2.red - rgb_sync2.blue) <= 6))then
              for i in 0 to 30 loop
                k_rgb(i).red <=   k_rgb_lut_0_l(i).max;   k_rgb(i).gre <=  k_rgb_lut_0_l(i).min;
                   k_rgb(i).blu <=  k_rgb_lut_0_l(i).mid;
```

```
2298                end loop;
2299
                ------------------
```

```
                    ------------------
```

```
                    ------------------------------------------------------------------------------------------------------------------
2402                    if (rgb_sync2.red = rgb_min1) then
2403                          for i in 0 to 30 loop
2404                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
2405                          end loop;
2406                    else
2407                          for i in 0 to 30 loop
2408                            k_rgb(i).red <=   k_rgb_lut_2_l(i).min;   k_rgb(i).gre <=
                                k_rgb_lut_2_l(i).mid;   k_rgb(i).blu <=  k_rgb_lut_2_l(i).max;
2409                          end loop;
2410                    end if;
2411                end if;
2412            end if;
2413
            ------------------------------------------------------------------------------------------------------------------
2414        end if;
2415    end if;
2416 end process;
2417 process (clk)begin
2418     if rising_edge(clk) then
2419       k1_rgb      <= k_rgb;
2420       k2_rgb      <= k1_rgb;
2421       k3_rgb      <= k2_rgb;
2422       k4_rgb      <= k3_rgb;
2423       k5_rgb      <= k4_rgb;
2424       k6_rgb      <= k5_rgb;
2425       k7_rgb      <= k6_rgb;
2426       k8_rgb      <= k7_rgb;
2427       k9_rgb      <= k8_rgb;
2428       k10rgb      <= k9_rgb;
2429       k11rgb      <= k10rgb;
2430       k12rgb      <= k11rgb;
2431       k13rgb      <= k12rgb;
2432       k14rgb      <= k13rgb;
2433       k15rgb      <= k14rgb;
2434       k16rgb      <= k15rgb;
2435       k17rgb      <= k16rgb;
2436       k18rgb      <= k17rgb;
2437       k19rgb      <= k18rgb;
2438       k20rgb      <= k19rgb;
2439       k21rgb      <= k20rgb;
2440       k22rgb      <= k21rgb;
2441       k23rgb      <= k22rgb;
2442       k24rgb      <= k23rgb;
2443       k25rgb      <= k24rgb;
2444       k26rgb      <= k25rgb;
2445       k27rgb      <= k26rgb;
2446     end if;
2447 end process;
2448 process (clk)begin
2449     if rising_edge(clk) then
2450         rgb(1).red     <= std_logic_vector(to_unsigned(k26rgb(1).red, 8)) & "00";
2451         rgb(1).gre     <= std_logic_vector(to_unsigned(k26rgb(1).gre, 8)) & "00";
2452         rgb(1).blu     <= std_logic_vector(to_unsigned(k26rgb(1).blu, 8)) & "00";
2453         rgb(2).red     <= std_logic_vector(to_unsigned(k26rgb(2).red, 8)) & "00";
2454         rgb(2).gre     <= std_logic_vector(to_unsigned(k26rgb(2).gre, 8)) & "00";
2455         rgb(2).blu     <= std_logic_vector(to_unsigned(k26rgb(2).blu, 8)) & "00";
2456         rgb(3).red     <= std_logic_vector(to_unsigned(k26rgb(3).red, 8)) & "00";
2457         rgb(3).gre     <= std_logic_vector(to_unsigned(k26rgb(3).gre, 8)) & "00";
2458         rgb(3).blu     <= std_logic_vector(to_unsigned(k26rgb(3).blu, 8)) & "00";
2459         rgb(4).red     <= std_logic_vector(to_unsigned(k26rgb(4).red, 8)) & "00";
2460         rgb(4).gre     <= std_logic_vector(to_unsigned(k26rgb(4).gre, 8)) & "00";
2461         rgb(4).blu     <= std_logic_vector(to_unsigned(k26rgb(4).blu, 8)) & "00";
2462         rgb(5).red     <= std_logic_vector(to_unsigned(k26rgb(5).red, 8)) & "00";
2463         rgb(5).gre     <= std_logic_vector(to_unsigned(k26rgb(5).gre, 8)) & "00";
2464         rgb(5).blu     <= std_logic_vector(to_unsigned(k26rgb(5).blu, 8)) & "00";
2465         rgb(6).red     <= std_logic_vector(to_unsigned(k26rgb(6).red, 8)) & "00";
2466         rgb(6).gre     <= std_logic_vector(to_unsigned(k26rgb(6).gre, 8)) & "00";
2467         rgb(6).blu     <= std_logic_vector(to_unsigned(k26rgb(6).blu, 8)) & "00";
2468         rgb(7).red     <= std_logic_vector(to_unsigned(k26rgb(7).red, 8)) & "00";
2469         rgb(7).gre     <= std_logic_vector(to_unsigned(k26rgb(7).gre, 8)) & "00";
```

```
2470            rgb(7).blu     <= std_logic_vector(to_unsigned(k26rgb(7).blu, 8)) & "00";
2471            rgb(8).red     <= std_logic_vector(to_unsigned(k26rgb(8).red, 8)) & "00";
2472            rgb(8).gre     <= std_logic_vector(to_unsigned(k26rgb(8).gre, 8)) & "00";
2473            rgb(8).blu     <= std_logic_vector(to_unsigned(k26rgb(8).blu, 8)) & "00";
2474            rgb(9).red     <= std_logic_vector(to_unsigned(k26rgb(9).red, 8)) & "00";
2475            rgb(9).gre     <= std_logic_vector(to_unsigned(k26rgb(9).gre, 8)) & "00";
2476            rgb(9).blu     <= std_logic_vector(to_unsigned(k26rgb(9).blu, 8)) & "00";
2477            rgb(10).red    <= std_logic_vector(to_unsigned(k26rgb(10).red, 8)) & "00";
2478            rgb(10).gre    <= std_logic_vector(to_unsigned(k26rgb(10).gre, 8)) & "00";
2479            rgb(10).blu    <= std_logic_vector(to_unsigned(k26rgb(10).blu, 8)) & "00";
2480            rgb(11).red    <= std_logic_vector(to_unsigned(k26rgb(11).red, 8)) & "00";
2481            rgb(11).gre    <= std_logic_vector(to_unsigned(k26rgb(11).gre, 8)) & "00";
2482            rgb(11).blu    <= std_logic_vector(to_unsigned(k26rgb(11).blu, 8)) & "00";
2483            rgb(12).red    <= std_logic_vector(to_unsigned(k26rgb(12).red, 8)) & "00";
2484            rgb(12).gre    <= std_logic_vector(to_unsigned(k26rgb(12).gre, 8)) & "00";
2485            rgb(12).blu    <= std_logic_vector(to_unsigned(k26rgb(12).blu, 8)) & "00";
2486            rgb(13).red    <= std_logic_vector(to_unsigned(k26rgb(13).red, 8)) & "00";
2487            rgb(13).gre    <= std_logic_vector(to_unsigned(k26rgb(13).gre, 8)) & "00";
2488            rgb(13).blu    <= std_logic_vector(to_unsigned(k26rgb(13).blu, 8)) & "00";
2489            rgb(14).red    <= std_logic_vector(to_unsigned(k26rgb(14).red, 8)) & "00";
2490            rgb(14).gre    <= std_logic_vector(to_unsigned(k26rgb(14).gre, 8)) & "00";
2491            rgb(14).blu    <= std_logic_vector(to_unsigned(k26rgb(14).blu, 8)) & "00";
2492            rgb(15).red    <= std_logic_vector(to_unsigned(k26rgb(15).red, 8)) & "00";
2493            rgb(15).gre    <= std_logic_vector(to_unsigned(k26rgb(15).gre, 8)) & "00";
2494            rgb(15).blu    <= std_logic_vector(to_unsigned(k26rgb(15).blu, 8)) & "00";
2495            rgb(16).red    <= std_logic_vector(to_unsigned(k26rgb(16).red, 8)) & "00";
2496            rgb(16).gre    <= std_logic_vector(to_unsigned(k26rgb(16).gre, 8)) & "00";
2497            rgb(16).blu    <= std_logic_vector(to_unsigned(k26rgb(16).blu, 8)) & "00";
2498            rgb(17).red    <= std_logic_vector(to_unsigned(k26rgb(17).red, 8)) & "00";
2499            rgb(17).gre    <= std_logic_vector(to_unsigned(k26rgb(17).gre, 8)) & "00";
2500            rgb(17).blu    <= std_logic_vector(to_unsigned(k26rgb(17).blu, 8)) & "00";
2501            rgb(18).red    <= std_logic_vector(to_unsigned(k26rgb(18).red, 8)) & "00";
2502            rgb(18).gre    <= std_logic_vector(to_unsigned(k26rgb(18).gre, 8)) & "00";
2503            rgb(18).blu    <= std_logic_vector(to_unsigned(k26rgb(18).blu, 8)) & "00";
2504            rgb(19).red    <= std_logic_vector(to_unsigned(k26rgb(19).red, 8)) & "00";
2505            rgb(19).gre    <= std_logic_vector(to_unsigned(k26rgb(19).gre, 8)) & "00";
2506            rgb(19).blu    <= std_logic_vector(to_unsigned(k26rgb(19).blu, 8)) & "00";
2507            rgb(20).red    <= std_logic_vector(to_unsigned(k26rgb(20).red, 8)) & "00";
2508            rgb(20).gre    <= std_logic_vector(to_unsigned(k26rgb(20).gre, 8)) & "00";
2509            rgb(20).blu    <= std_logic_vector(to_unsigned(k26rgb(20).blu, 8)) & "00";
2510            rgb(21).red    <= std_logic_vector(to_unsigned(k26rgb(21).red, 8)) & "00";
2511            rgb(21).gre    <= std_logic_vector(to_unsigned(k26rgb(21).gre, 8)) & "00";
2512            rgb(21).blu    <= std_logic_vector(to_unsigned(k26rgb(21).blu, 8)) & "00";
2513            rgb(22).red    <= std_logic_vector(to_unsigned(k26rgb(22).red, 8)) & "00";
2514            rgb(22).gre    <= std_logic_vector(to_unsigned(k26rgb(22).gre, 8)) & "00";
2515            rgb(22).blu    <= std_logic_vector(to_unsigned(k26rgb(22).blu, 8)) & "00";
2516            rgb(23).red    <= std_logic_vector(to_unsigned(k26rgb(23).red, 8)) & "00";
2517            rgb(23).gre    <= std_logic_vector(to_unsigned(k26rgb(23).gre, 8)) & "00";
2518            rgb(23).blu    <= std_logic_vector(to_unsigned(k26rgb(23).blu, 8)) & "00";
2519            rgb(24).red    <= std_logic_vector(to_unsigned(k26rgb(24).red, 8)) & "00";
2520            rgb(24).gre    <= std_logic_vector(to_unsigned(k26rgb(24).gre, 8)) & "00";
2521            rgb(24).blu    <= std_logic_vector(to_unsigned(k26rgb(24).blu, 8)) & "00";
2522            rgb(25).red    <= std_logic_vector(to_unsigned(k26rgb(25).red, 8)) & "00";
2523            rgb(25).gre    <= std_logic_vector(to_unsigned(k26rgb(25).gre, 8)) & "00";
2524            rgb(25).blu    <= std_logic_vector(to_unsigned(k26rgb(25).blu, 8)) & "00";
2525            rgb(26).red    <= std_logic_vector(to_unsigned(k26rgb(26).red, 8)) & "00";
2526            rgb(26).gre    <= std_logic_vector(to_unsigned(k26rgb(26).gre, 8)) & "00";
2527            rgb(26).blu    <= std_logic_vector(to_unsigned(k26rgb(26).blu, 8)) & "00";
2528            rgb(27).red    <= std_logic_vector(to_unsigned(k26rgb(27).red, 8)) & "00";
2529            rgb(27).gre    <= std_logic_vector(to_unsigned(k26rgb(27).gre, 8)) & "00";
2530            rgb(27).blu    <= std_logic_vector(to_unsigned(k26rgb(27).blu, 8)) & "00";
2531            rgb(28).red    <= std_logic_vector(to_unsigned(k26rgb(28).red, 8)) & "00";
2532            rgb(28).gre    <= std_logic_vector(to_unsigned(k26rgb(28).gre, 8)) & "00";
2533            rgb(28).blu    <= std_logic_vector(to_unsigned(k26rgb(28).blu, 8)) & "00";
2534            rgb(29).red    <= std_logic_vector(to_unsigned(k26rgb(29).red, 8)) & "00";
2535            rgb(29).gre    <= std_logic_vector(to_unsigned(k26rgb(29).gre, 8)) & "00";
2536            rgb(29).blu    <= std_logic_vector(to_unsigned(k26rgb(29).blu, 8)) & "00";
2537            rgb(30).red    <= std_logic_vector(to_unsigned(k26rgb(30).red, 8)) & "00";
2538            rgb(30).gre    <= std_logic_vector(to_unsigned(k26rgb(30).gre, 8)) & "00";
2539            rgb(30).blu    <= std_logic_vector(to_unsigned(k26rgb(30).blu, 8)) & "00";
2540        end if;
2541    end process;
2542    color_k1_clustering_inst: rgb_cluster_core
2543    generic map(
```

```
2544        data_width        => i_data_width)
2545    port map(
2546        clk                => clk,
2547        rst_n              => rst_n,
2548        pixel_in_rgb               => rgb_std4sync,
2549        k_rgb.red          => k_rgb(1).red,
2550        k_rgb.gre          => k_rgb(1).gre,
2551        k_rgb.blu          => k_rgb(1).blu,
2552        threshold          => thr1.threshold1);
2553    color_k2_clustering_inst: rgb_cluster_core
2554    generic map(
2555        data_width        => i_data_width)
2556    port map(
2557        clk                => clk,
2558        rst_n              => rst_n,
2559        pixel_in_rgb               => rgb_std4sync,
2560        k_rgb.red          => k_rgb(2).red,
2561        k_rgb.gre          => k_rgb(2).gre,
2562        k_rgb.blu          => k_rgb(2).blu,
2563        threshold          => thr1.threshold2);
2564    color_k3_clustering_inst: rgb_cluster_core
2565    generic map(
2566        data_width        => i_data_width)
2567    port map(
2568        clk                => clk,
2569        rst_n              => rst_n,
2570        pixel_in_rgb               => rgb_std4sync,
2571        k_rgb.red          => k_rgb(3).red,
2572        k_rgb.gre          => k_rgb(3).gre,
2573        k_rgb.blu          => k_rgb(3).blu,
2574        threshold          => thr1.threshold3);
2575    color_k4_clustering_inst: rgb_cluster_core
2576    generic map(
2577        data_width        => i_data_width)
2578    port map(
2579        clk                => clk,
2580        rst_n              => rst_n,
2581        pixel_in_rgb               => rgb_std4sync,
2582        k_rgb.red          => k_rgb(4).red,
2583        k_rgb.gre          => k_rgb(4).gre,
2584        k_rgb.blu          => k_rgb(4).blu,
2585        threshold          => thr1.threshold4);
2586    color_k5_clustering_inst: rgb_cluster_core
2587    generic map(
2588        data_width        => i_data_width)
2589    port map(
2590        clk                => clk,
2591        rst_n              => rst_n,
2592        pixel_in_rgb               => rgb_std4sync,
2593        k_rgb.red          => k_rgb(5).red,
2594        k_rgb.gre          => k_rgb(5).gre,
2595        k_rgb.blu          => k_rgb(5).blu,
2596        threshold          => thr1.threshold5);
2597    color_k6_clustering_inst: rgb_cluster_core
2598    generic map(
2599        data_width        => i_data_width)
2600    port map(
2601        clk                => clk,
2602        rst_n              => rst_n,
2603        pixel_in_rgb               => rgb_std4sync,
2604        k_rgb.red          => k_rgb(6).red,
2605        k_rgb.gre          => k_rgb(6).gre,
2606        k_rgb.blu          => k_rgb(6).blu,
2607        threshold          => thr1.threshold6);
2608    color_k7_clustering_inst: rgb_cluster_core
2609    generic map(
2610        data_width        => i_data_width)
2611    port map(
2612        clk                => clk,
2613        rst_n              => rst_n,
2614        pixel_in_rgb               => rgb_std4sync,
2615        k_rgb.red          => k_rgb(7).red,
2616        k_rgb.gre          => k_rgb(7).gre,
2617        k_rgb.blu          => k_rgb(7).blu,
```

```
2618        threshold          => thr1.threshold7);
2619    color_k8_clustering_inst: rgb_cluster_core
2620    generic map(
2621        data_width        => i_data_width)
2622    port map(
2623        clk                => clk,
2624        rst_n              => rst_n,
2625        pixel_in_rgb               => rgb_std4sync,
2626        k_rgb.red          => k_rgb(8).red,
2627        k_rgb.gre          => k_rgb(8).gre,
2628        k_rgb.blu          => k_rgb(8).blu,
2629        threshold          => thr1.threshold8);
2630    color_k9_clustering_inst: rgb_cluster_core
2631    generic map(
2632        data_width        => i_data_width)
2633    port map(
2634        clk                => clk,
2635        rst_n              => rst_n,
2636        pixel_in_rgb               => rgb_std4sync,
2637        k_rgb.red          => k_rgb(9).red,
2638        k_rgb.gre          => k_rgb(9).gre,
2639        k_rgb.blu          => k_rgb(9).blu,
2640        threshold          => thr1.threshold9);
2641    color_k10_clustering_inst: rgb_cluster_core
2642    generic map(
2643        data_width        => i_data_width)
2644    port map(
2645        clk                => clk,
2646        rst_n              => rst_n,
2647        pixel_in_rgb               => rgb_std4sync,
2648        k_rgb.red          => k_rgb(10).red,
2649        k_rgb.gre          => k_rgb(10).gre,
2650        k_rgb.blu          => k_rgb(10).blu,
2651        threshold          => thr1.threshold10);
2652    color_k11_clustering_inst: rgb_cluster_core
2653    generic map(
2654        data_width        => i_data_width)
2655    port map(
2656        clk                => clk,
2657        rst_n              => rst_n,
2658        pixel_in_rgb               => rgb_std4sync,
2659        k_rgb.red          => k_rgb(11).red,
2660        k_rgb.gre          => k_rgb(11).gre,
2661        k_rgb.blu          => k_rgb(11).blu,
2662        threshold          => thr1.threshold11);
2663    color_k12_clustering_inst: rgb_cluster_core
2664    generic map(
2665        data_width        => i_data_width)
2666    port map(
2667        clk                => clk,
2668        rst_n              => rst_n,
2669        pixel_in_rgb               => rgb_std4sync,
2670        k_rgb.red          => k_rgb(12).red,
2671        k_rgb.gre          => k_rgb(12).gre,
2672        k_rgb.blu          => k_rgb(12).blu,
2673        threshold          => thr1.threshold12);
2674    color_k13_clustering_inst: rgb_cluster_core
2675    generic map(
2676        data_width        => i_data_width)
2677    port map(
2678        clk                => clk,
2679        rst_n              => rst_n,
2680        pixel_in_rgb               => rgb_std4sync,
2681        k_rgb.red          => k_rgb(13).red,
2682        k_rgb.gre          => k_rgb(13).gre,
2683        k_rgb.blu          => k_rgb(13).blu,
2684        threshold          => thr1.threshold13);
2685    color_k14_clustering_inst: rgb_cluster_core
2686    generic map(
2687        data_width        => i_data_width)
2688    port map(
2689        clk                => clk,
2690        rst_n              => rst_n,
2691        pixel_in_rgb               => rgb_std4sync,
```

```
2692        k_rgb.red          => k_rgb(14).red,
2693        k_rgb.gre          => k_rgb(14).gre,
2694        k_rgb.blu          => k_rgb(14).blu,
2695        threshold          => thr1.threshold14);
2696    color_k15_clustering_inst: rgb_cluster_core
2697    generic map(
2698        data_width        => i_data_width)
2699    port map(
2700        clk                => clk,
2701        rst_n              => rst_n,
2702        pixel_in_rgb               => rgb_std4sync,
2703        k_rgb.red          => k_rgb(15).red,
2704        k_rgb.gre          => k_rgb(15).gre,
2705        k_rgb.blu          => k_rgb(15).blu,
2706        threshold          => thr1.threshold15);
2707    color_k16clustering_inst: rgb_cluster_core
2708    generic map(
2709        data_width        => i_data_width)
2710    port map(
2711        clk                => clk,
2712        rst_n              => rst_n,
2713        pixel_in_rgb               => rgb_std4sync,
2714        k_rgb.red          => k_rgb(16).red,
2715        k_rgb.gre          => k_rgb(16).gre,
2716        k_rgb.blu          => k_rgb(16).blu,
2717        threshold          => thr1.threshold16);
2718    color_k17_clustering_inst: rgb_cluster_core
2719    generic map(
2720        data_width        => i_data_width)
2721    port map(
2722        clk                => clk,
2723        rst_n              => rst_n,
2724        pixel_in_rgb               => rgb_std4sync,
2725        k_rgb.red          => k_rgb(17).red,
2726        k_rgb.gre          => k_rgb(17).gre,
2727        k_rgb.blu          => k_rgb(17).blu,
2728        threshold          => thr1.threshold17);
2729    color_k18_clustering_inst: rgb_cluster_core
2730    generic map(
2731        data_width        => i_data_width)
2732    port map(
2733        clk                => clk,
2734        rst_n              => rst_n,
2735        pixel_in_rgb               => rgb_std4sync,
2736        k_rgb.red          => k_rgb(18).red,
2737        k_rgb.gre          => k_rgb(18).gre,
2738        k_rgb.blu          => k_rgb(18).blu,
2739        threshold          => thr1.threshold18);
2740    color_k19_clustering_inst: rgb_cluster_core
2741    generic map(
2742        data_width        => i_data_width)
2743    port map(
2744        clk                => clk,
2745        rst_n              => rst_n,
2746        pixel_in_rgb               => rgb_std4sync,
2747        k_rgb.red          => k_rgb(19).red,
2748        k_rgb.gre          => k_rgb(19).gre,
2749        k_rgb.blu          => k_rgb(19).blu,
2750        threshold          => thr1.threshold19);
2751    color_k20_clustering_inst: rgb_cluster_core
2752    generic map(
2753        data_width        => i_data_width)
2754    port map(
2755        clk                => clk,
2756        rst_n              => rst_n,
2757        pixel_in_rgb               => rgb_std4sync,
2758        k_rgb.red          => k_rgb(20).red,
2759        k_rgb.gre          => k_rgb(20).gre,
2760        k_rgb.blu          => k_rgb(20).blu,
2761        threshold          => thr1.threshold20);
2762    color_k21_clustering_inst: rgb_cluster_core
2763    generic map(
2764        data_width        => i_data_width)
2765    port map(
```

```
2766        clk                 => clk,
2767        rst_n               => rst_n,
2768        pixel_in_rgb                => rgb_std4sync,
2769        k_rgb.red           => k_rgb(21).red,
2770        k_rgb.gre           => k_rgb(21).gre,
2771        k_rgb.blu           => k_rgb(21).blu,
2772        threshold           => thr1.threshold21);
2773    color_k22_clustering_inst: rgb_cluster_core
2774    generic map(
2775        data_width        => i_data_width)
2776    port map(
2777        clk                 => clk,
2778        rst_n               => rst_n,
2779        pixel_in_rgb                => rgb_std4sync,
2780        k_rgb.red           => k_rgb(22).red,
2781        k_rgb.gre           => k_rgb(22).gre,
2782        k_rgb.blu           => k_rgb(22).blu,
2783        threshold           => thr1.threshold22);
2784    color_k23_clustering_inst: rgb_cluster_core
2785    generic map(
2786        data_width        => i_data_width)
2787    port map(
2788        clk                 => clk,
2789        rst_n               => rst_n,
2790        pixel_in_rgb                => rgb_std4sync,
2791        k_rgb.red           => k_rgb(23).red,
2792        k_rgb.gre           => k_rgb(23).gre,
2793        k_rgb.blu           => k_rgb(23).blu,
2794        threshold           => thr1.threshold23);
2795    color_k24_clustering_inst: rgb_cluster_core
2796    generic map(
2797        data_width        => i_data_width)
2798    port map(
2799        clk                 => clk,
2800        rst_n               => rst_n,
2801        pixel_in_rgb                => rgb_std4sync,
2802        k_rgb.red           => k_rgb(24).red,
2803        k_rgb.gre           => k_rgb(24).gre,
2804        k_rgb.blu           => k_rgb(24).blu,
2805        threshold           => thr1.threshold24);
2806    color_k25_clustering_inst: rgb_cluster_core
2807    generic map(
2808        data_width        => i_data_width)
2809    port map(
2810        clk                 => clk,
2811        rst_n               => rst_n,
2812        pixel_in_rgb                => rgb_std4sync,
2813        k_rgb.red           => k_rgb(25).red,
2814        k_rgb.gre           => k_rgb(25).gre,
2815        k_rgb.blu           => k_rgb(25).blu,
2816        threshold           => thr1.threshold25);
2817    color_k26_clustering_inst: rgb_cluster_core
2818    generic map(
2819        data_width        => i_data_width)
2820    port map(
2821        clk                 => clk,
2822        rst_n               => rst_n,
2823        pixel_in_rgb                => rgb_std4sync,
2824        k_rgb.red           => k_rgb(26).red,
2825        k_rgb.gre           => k_rgb(26).gre,
2826        k_rgb.blu           => k_rgb(26).blu,
2827        threshold           => thr1.threshold26);
2828    color_k27_clustering_inst: rgb_cluster_core
2829    generic map(
2830        data_width        => i_data_width)
2831    port map(
2832        clk                 => clk,
2833        rst_n               => rst_n,
2834        pixel_in_rgb                => rgb_std4sync,
2835        k_rgb.red           => k_rgb(27).red,
2836        k_rgb.gre           => k_rgb(27).gre,
2837        k_rgb.blu           => k_rgb(27).blu,
2838        threshold           => thr1.threshold27);
2839    color_k28_clustering_inst: rgb_cluster_core
```

```
2840  generic map(
2841      data_width        => i_data_width)
2842  port map(
2843      clk                => clk,
2844      rst_n              => rst_n,
2845      pixel_in_rgb               => rgb_std4sync,
2846      k_rgb.red          => k_rgb(28).red,
2847      k_rgb.gre          => k_rgb(28).gre,
2848      k_rgb.blu          => k_rgb(28).blu,
2849      threshold          => thr1.threshold28);
2850  color_k29_clustering_inst: rgb_cluster_core
2851  generic map(
2852      data_width        => i_data_width)
2853  port map(
2854      clk                => clk,
2855      rst_n              => rst_n,
2856      pixel_in_rgb        => rgb_std4sync,
2857      k_rgb.red          => k_rgb(29).red,
2858      k_rgb.gre          => k_rgb(29).gre,
2859      k_rgb.blu          => k_rgb(29).blu,
2860      threshold          => thr1.threshold29);
2861  color_k30_clustering_inst: rgb_cluster_core
2862  generic map(
2863      data_width        => i_data_width)
2864  port map(
2865      clk                => clk,
2866      rst_n              => rst_n,
2867      pixel_in_rgb               => rgb_std4sync,
2868      k_rgb.red          => k_rgb(30).red,
2869      k_rgb.gre          => k_rgb(30).gre,
2870      k_rgb.blu          => k_rgb(30).blu,
2871      threshold          => thr1.threshold30);
2872  process (clk) begin
2873      if rising_edge(clk) then
2874          threshold_lms_1  <= int_min_val(thr1.threshold1,thr1.threshold2,thr1.threshold3,thr1.
              threshold4,thr1.threshold5);
2875          threshold_lms_2  <= int_min_val(thr1.threshold6,thr1.threshold7,thr1.threshold8,thr1.
              threshold9,thr1.threshold10);
2876          threshold_lms_3  <= int_min_val(thr1.threshold11,thr1.threshold12,thr1.threshold13,thr1.
              threshold14,thr1.threshold15);
2877          threshold_lms_4  <= int_min_val(thr1.threshold16,thr1.threshold17,thr1.threshold18,thr1.
              threshold19,thr1.threshold20);
2878          threshold_lms_5  <= int_min_val(thr1.threshold21,thr1.threshold22,thr1.threshold23,thr1.
              threshold24,thr1.threshold25);
2879          threshold_lms_6  <= int_min_val(thr1.threshold26,thr1.threshold27,thr1.threshold28,thr1.
              threshold29,thr1.threshold30);
2880      end if;
2881  end process;
2882  process (clk) begin
2883      if rising_edge(clk) then
2884          threshold_red1   <= int_min_val(threshold_lms_1,threshold_lms_2,threshold_lms_3);
2885          threshold_red2   <= int_min_val(threshold_lms_4,threshold_lms_5,threshold_lms_6);
2886      end if;
2887  end process;
2888  process (clk) begin
2889      if rising_edge(clk) then
2890          threshold_lms    <= int_min_val(threshold_red1,threshold_red2);
2891          threshold        <= threshold_lms;
2892      end if;
2893  end process;
2894  process (clk) begin
2895      if rising_edge(clk) then
2896          thr2 <= thr1;
2897          thr3 <= thr2;
2898          thr4 <= thr3;
2899          thr5 <= thr4;
2900      end if;
2901  end process;
2902  process (clk) begin
2903      if rising_edge(clk) then
2904          if ((thr5.threshold1  = threshold)) then
2905              pixel_out_rgb.red     <= rgb(1).red;
2906              pixel_out_rgb.green   <= rgb(1).gre;
2907              pixel_out_rgb.blue    <= rgb(1).blu;
```

```
2908            elsif((thr5.threshold2 = threshold)) then
2909                pixel_out_rgb.red     <= rgb(2).red;
2910                pixel_out_rgb.green   <= rgb(2).gre;
2911                pixel_out_rgb.blue    <= rgb(2).blu;
2912            elsif((thr5.threshold3 = threshold)) then
2913                pixel_out_rgb.red     <= rgb(3).red;
2914                pixel_out_rgb.green   <= rgb(3).gre;
2915                pixel_out_rgb.blue    <= rgb(3).blu;
2916            elsif((thr5.threshold4 = threshold)) then
2917                pixel_out_rgb.red     <= rgb(4).red;
2918                pixel_out_rgb.green   <= rgb(4).gre;
2919                pixel_out_rgb.blue    <= rgb(4).blu;
2920            elsif((thr5.threshold5 = threshold)) then
2921                pixel_out_rgb.red     <= rgb(5).red;
2922                pixel_out_rgb.green   <= rgb(5).gre;
2923                pixel_out_rgb.blue    <= rgb(5).blu;
2924            elsif((thr5.threshold6 = threshold)) then
2925                pixel_out_rgb.red     <= rgb(6).red;
2926                pixel_out_rgb.green   <= rgb(6).gre;
2927                pixel_out_rgb.blue    <= rgb(6).blu;
2928            elsif((thr5.threshold7 = threshold)) then
2929                pixel_out_rgb.red     <= rgb(7).red;
2930                pixel_out_rgb.green   <= rgb(7).gre;
2931                pixel_out_rgb.blue    <= rgb(7).blu;
2932            elsif((thr5.threshold8 = threshold)) then
2933                pixel_out_rgb.red     <= rgb(8).red;
2934                pixel_out_rgb.green   <= rgb(8).gre;
2935                pixel_out_rgb.blue    <= rgb(8).blu;
2936            elsif((thr5.threshold9 = threshold)) then
2937                pixel_out_rgb.red     <= rgb(9).red;
2938                pixel_out_rgb.green   <= rgb(9).gre;
2939                pixel_out_rgb.blue    <= rgb(9).blu;
2940            elsif((thr5.threshold10 = threshold)) then
2941                pixel_out_rgb.red     <= rgb(10).red;
2942                pixel_out_rgb.green   <= rgb(10).gre;
2943                pixel_out_rgb.blue    <= rgb(10).blu;
2944            elsif((thr5.threshold11 = threshold)) then
2945                pixel_out_rgb.red     <= rgb(11).red;
2946                pixel_out_rgb.green   <= rgb(11).gre;
2947                pixel_out_rgb.blue    <= rgb(11).blu;
2948            elsif((thr5.threshold12 = threshold)) then
2949                pixel_out_rgb.red     <= rgb(12).red;
2950                pixel_out_rgb.green   <= rgb(12).gre;
2951                pixel_out_rgb.blue    <= rgb(12).blu;
2952            elsif((thr5.threshold13 = threshold)) then
2953                pixel_out_rgb.red     <= rgb(13).red;
2954                pixel_out_rgb.green   <= rgb(13).gre;
2955                pixel_out_rgb.blue    <= rgb(13).blu;
2956            elsif((thr5.threshold14 = threshold)) then
2957                pixel_out_rgb.red     <= rgb(14).red;
2958                pixel_out_rgb.green   <= rgb(14).gre;
2959                pixel_out_rgb.blue    <= rgb(14).blu;
2960            elsif((thr5.threshold15 = threshold)) then
2961                pixel_out_rgb.red     <= rgb(15).red;
2962                pixel_out_rgb.green   <= rgb(15).gre;
2963                pixel_out_rgb.blue    <= rgb(15).blu;
2964            elsif((thr5.threshold16 = threshold)) then
2965                pixel_out_rgb.red     <= rgb(16).red;
2966                pixel_out_rgb.green   <= rgb(16).gre;
2967                pixel_out_rgb.blue    <= rgb(16).blu;
2968            elsif((thr5.threshold17 = threshold)) then
2969                pixel_out_rgb.red     <= rgb(17).red;
2970                pixel_out_rgb.green   <= rgb(17).gre;
2971                pixel_out_rgb.blue    <= rgb(17).blu;
2972            elsif((thr5.threshold18 = threshold)) then
2973                pixel_out_rgb.red     <= rgb(18).red;
2974                pixel_out_rgb.green   <= rgb(18).gre;
2975                pixel_out_rgb.blue    <= rgb(18).blu;
2976            elsif((thr5.threshold19 = threshold)) then
2977                pixel_out_rgb.red     <= rgb(19).red;
2978                pixel_out_rgb.green   <= rgb(19).gre;
2979                pixel_out_rgb.blue    <= rgb(19).blu;
2980            elsif((thr5.threshold20 = threshold)) then
2981                pixel_out_rgb.red     <= rgb(20).red;
```

```
2982                pixel_out_rgb.green   <= rgb(20).gre;
2983                pixel_out_rgb.blue    <= rgb(20).blu;
2984            elsif((thr5.threshold21 = threshold)) then
2985                pixel_out_rgb.red     <= rgb(21).red;
2986                pixel_out_rgb.green   <= rgb(21).gre;
2987                pixel_out_rgb.blue    <= rgb(21).blu;
2988            elsif((thr5.threshold22 = threshold)) then
2989                pixel_out_rgb.red     <= rgb(22).red;
2990                pixel_out_rgb.green   <= rgb(22).gre;
2991                pixel_out_rgb.blue    <= rgb(22).blu;
2992            elsif((thr5.threshold23 = threshold)) then
2993                pixel_out_rgb.red     <= rgb(23).red;
2994                pixel_out_rgb.green   <= rgb(23).gre;
2995                pixel_out_rgb.blue    <= rgb(23).blu;
2996            elsif((thr5.threshold24 = threshold)) then
2997                pixel_out_rgb.red     <= rgb(24).red;
2998                pixel_out_rgb.green   <= rgb(24).gre;
2999                pixel_out_rgb.blue    <= rgb(24).blu;
3000            elsif((thr5.threshold25 = threshold)) then
3001                pixel_out_rgb.red     <= rgb(25).red;
3002                pixel_out_rgb.green   <= rgb(25).gre;
3003                pixel_out_rgb.blue    <= rgb(25).blu;
3004            elsif((thr5.threshold26 = threshold)) then
3005                pixel_out_rgb.red     <= rgb(26).red;
3006                pixel_out_rgb.green   <= rgb(26).gre;
3007                pixel_out_rgb.blue    <= rgb(26).blu;
3008            elsif((thr5.threshold27 = threshold)) then
3009                pixel_out_rgb.red     <= rgb(27).red;
3010                pixel_out_rgb.green   <= rgb(27).gre;
3011                pixel_out_rgb.blue    <= rgb(27).blu;
3012            elsif((thr5.threshold28 = threshold)) then
3013                pixel_out_rgb.red     <= rgb(28).red;
3014                pixel_out_rgb.green   <= rgb(28).gre;
3015                pixel_out_rgb.blue    <= rgb(28).blu;
3016            elsif((thr5.threshold29 = threshold)) then
3017                pixel_out_rgb.red     <= rgb(29).red;
3018                pixel_out_rgb.green   <= rgb(29).gre;
3019                pixel_out_rgb.blue    <= rgb(29).blu;
3020            elsif((thr5.threshold30 = threshold)) then
3021                pixel_out_rgb.red     <= rgb(30).red;
3022                pixel_out_rgb.green   <= rgb(30).gre;
3023                pixel_out_rgb.blue    <= rgb(30).blu;
3024            else
3025                pixel_out_rgb.red     <= (others => '1');
3026                pixel_out_rgb.green   <= (others => '1');
3027                pixel_out_rgb.blue    <= (others => '1');
3028            end if;
3029        end if;
3030    end process;
3031    pixel_out_rgb.valid <= rgbSyncValid(25);
3032    pixel_out_rgb.eol   <= rgbSyncEol(25);
3033    pixel_out_rgb.sof   <= rgbSyncSof(25);
3034    pixel_out_rgb.eof   <= rgbSyncEof(25);
3035    end architecture;
```